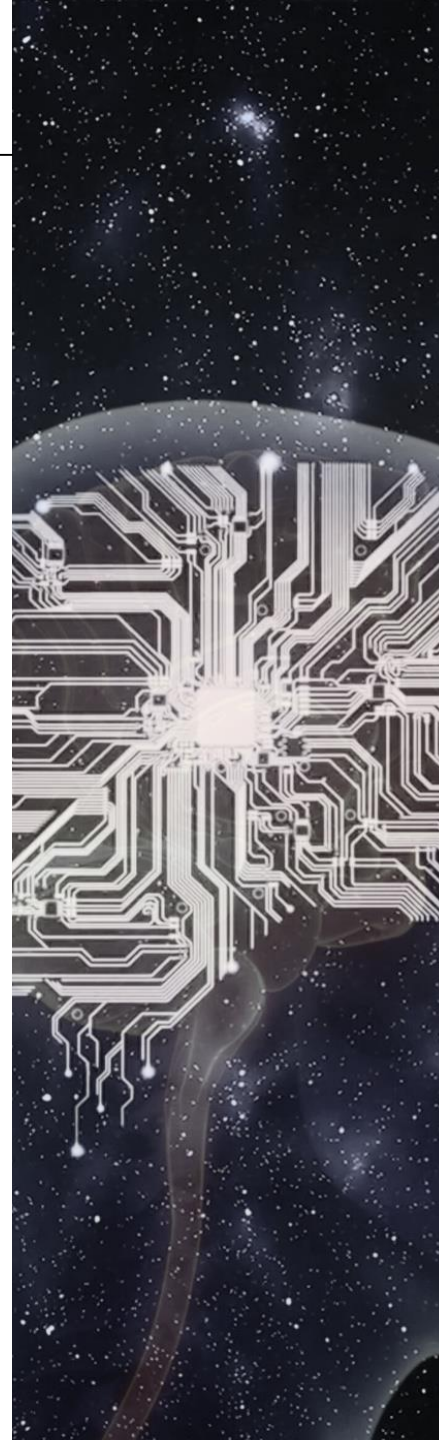


어셈블리 프로그래밍

2. x86 Processor architecture



Understanding x86 Assembly Language

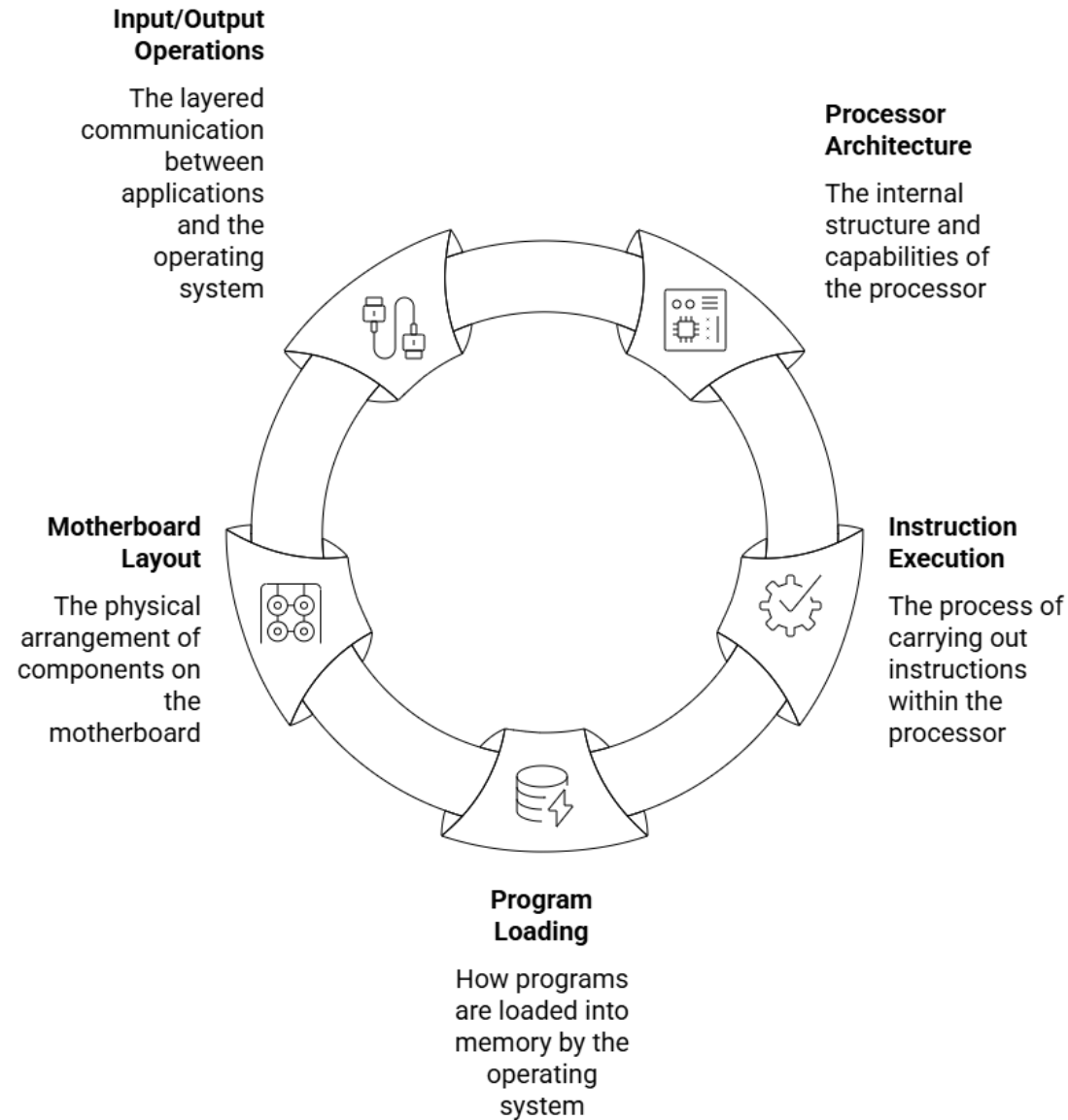
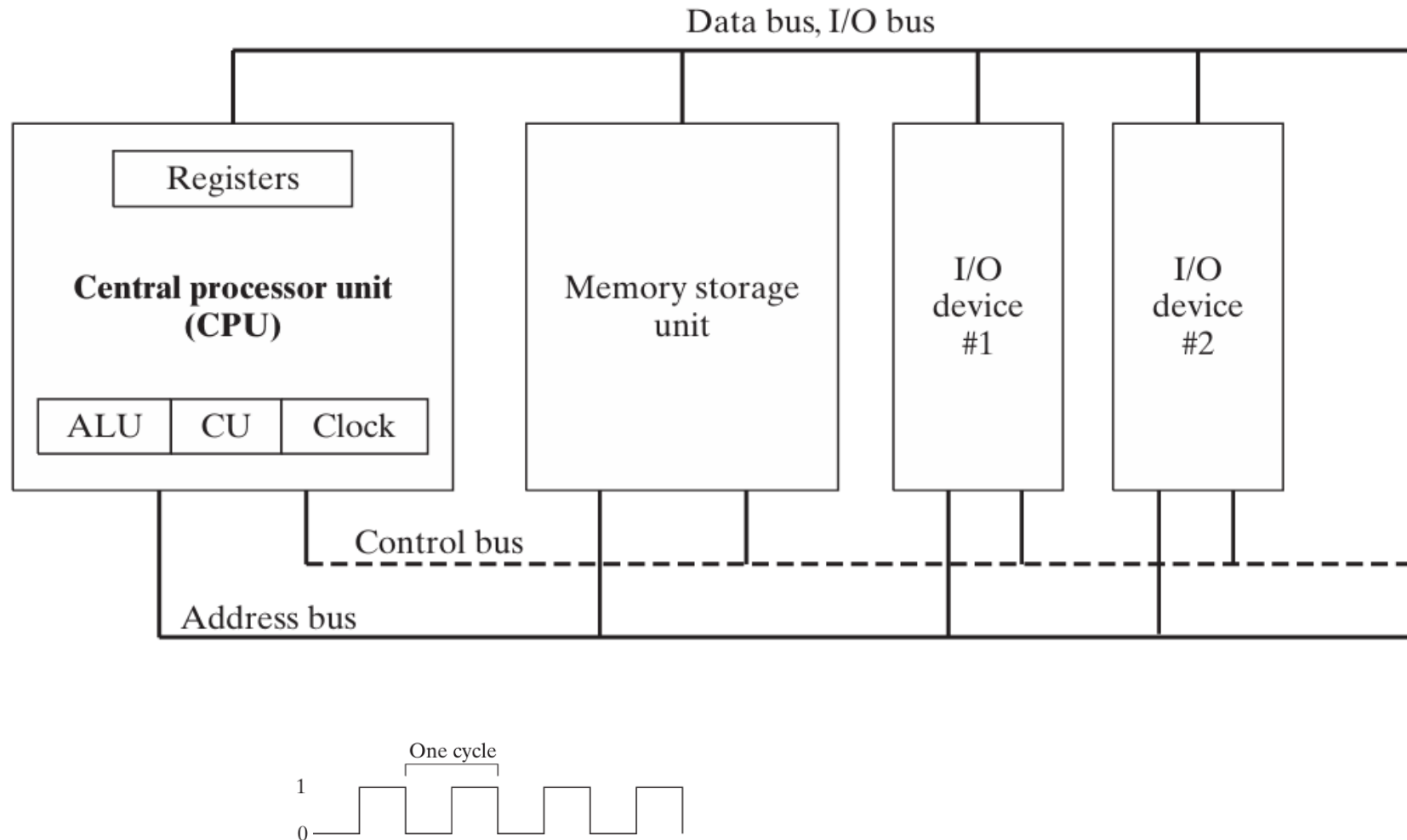
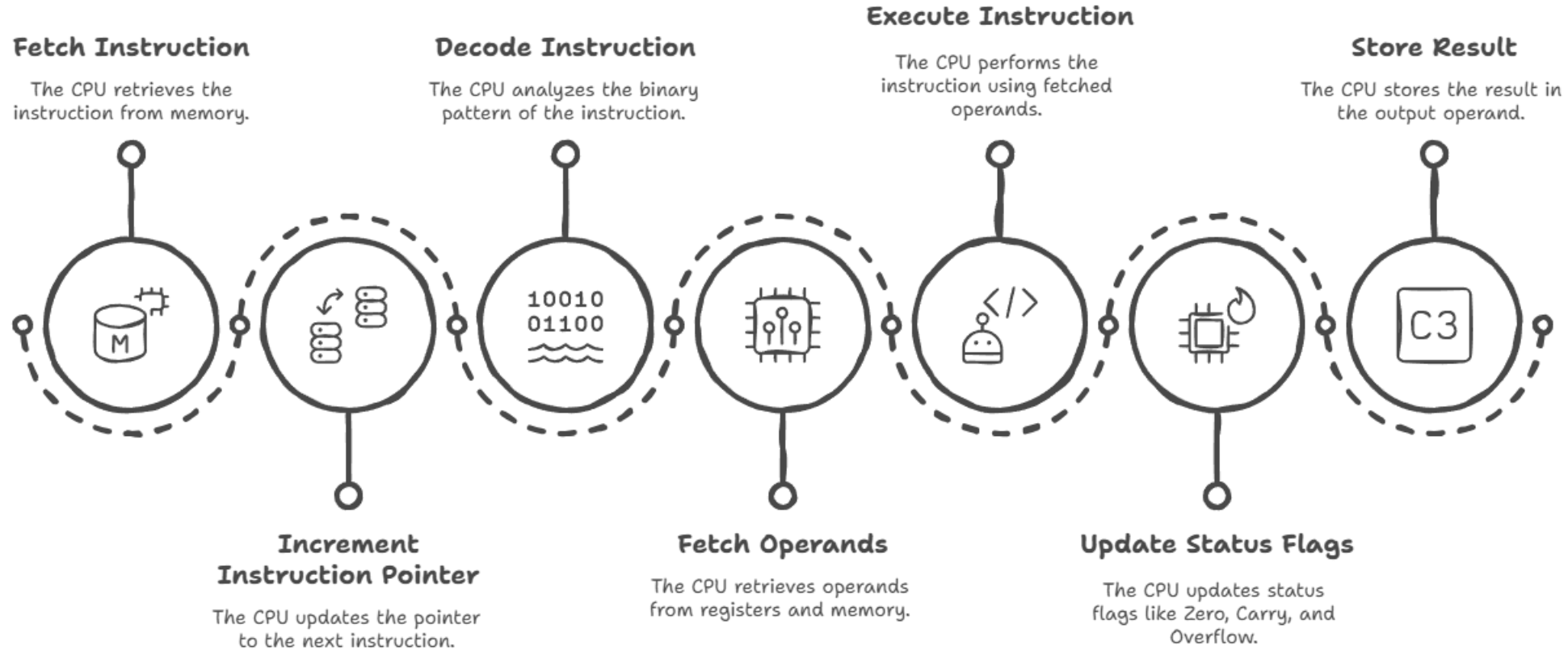


FIGURE 2-1 Block diagram of a microcomputer.



CPU Instruction Execution Sequence



Made with  Napkin

Instruction Decode Timeline (예: `MOV AX, 1234h` = `B8 34 12`)

1. Fetch B8

- IP = 1000 → 1001
- 첫 바이트(`B8`) 가져오기

2. Decode opcode (B8)

- CPU는 `MOV AX, imm16` 명령임을 인식
- → 즉시 2바이트 operand 필요하다는 걸 알게 됨

3. Fetch 34

- IP = 1001 → 1002
- 첫 번째 operand 바이트 가져오기

4. Fetch 12

- IP = 1002 → 1003
- 두 번째 operand 바이트 가져오기

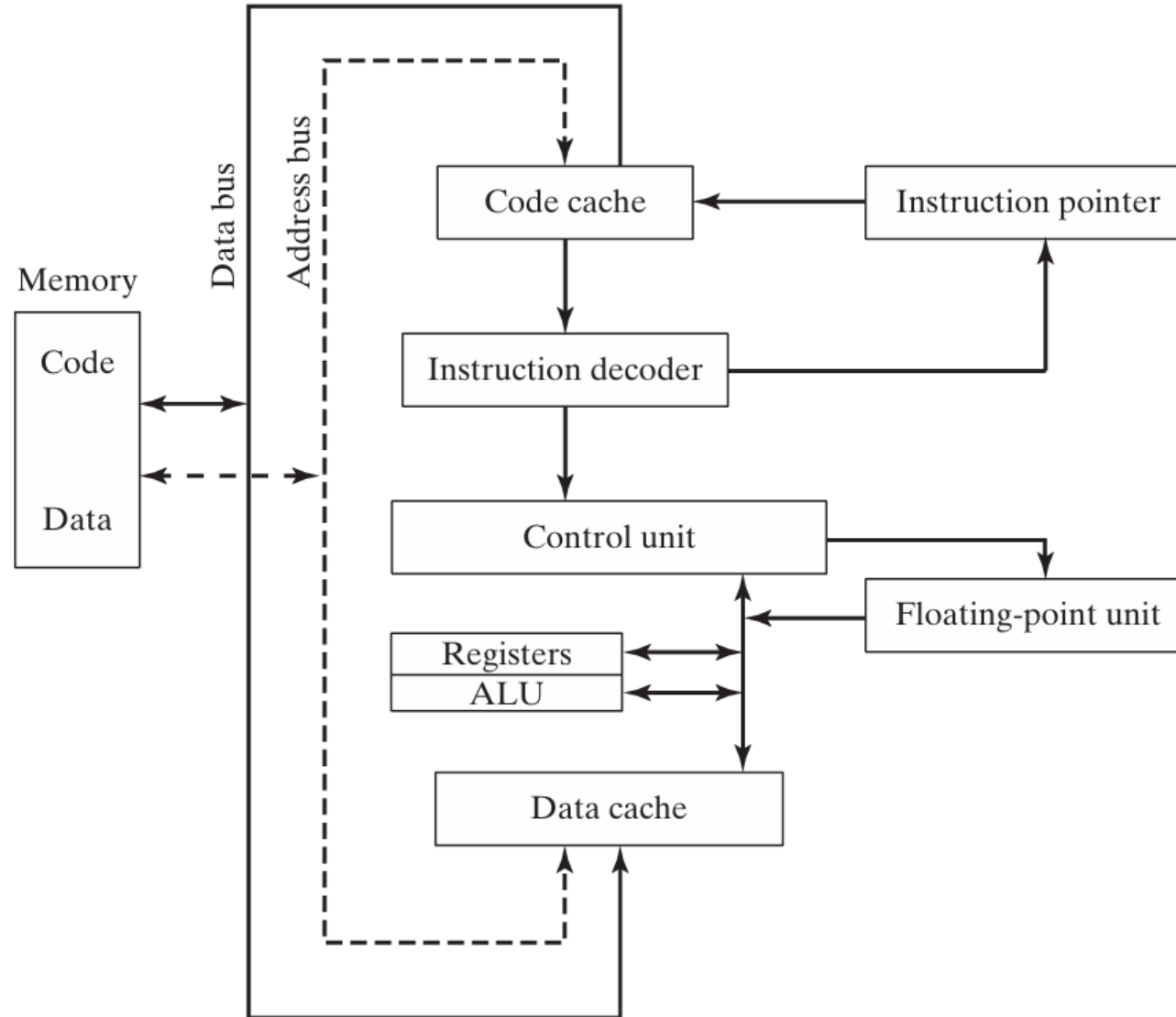
5. Instruction complete

- 이제 `B8 34 12` 전체 명령어 해독 완료
- `MOV AX, 1234h` 라고 확정

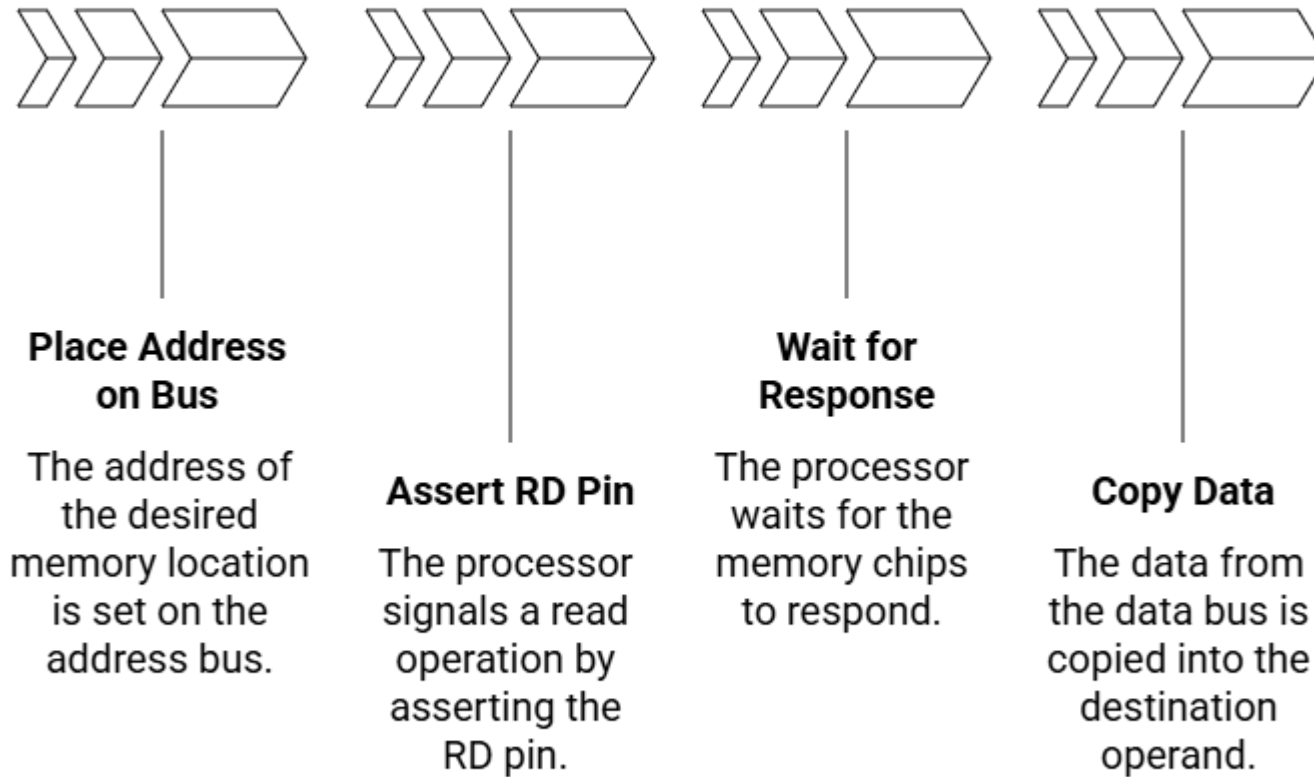
6. Execute

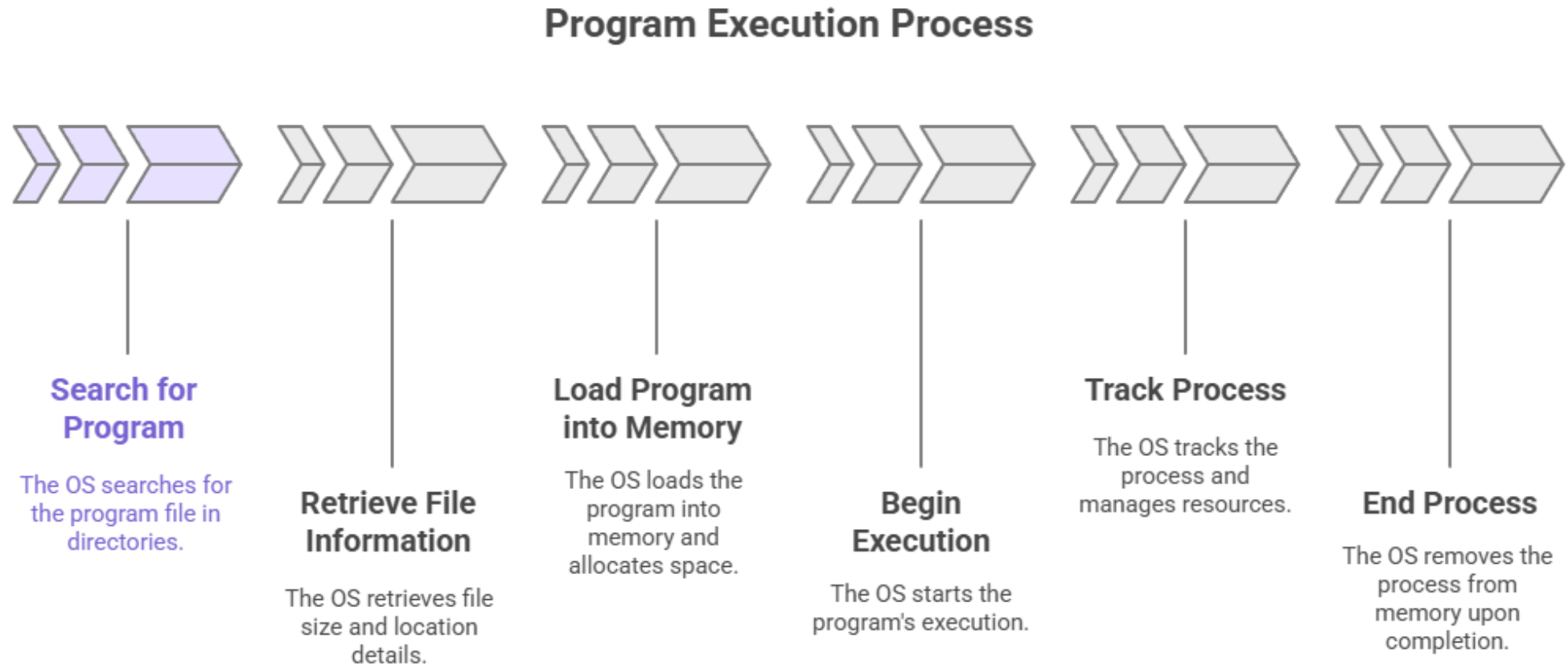
- $AX \leftarrow 1234h$

FIGURE 2-2 Simplified CPU block diagram.

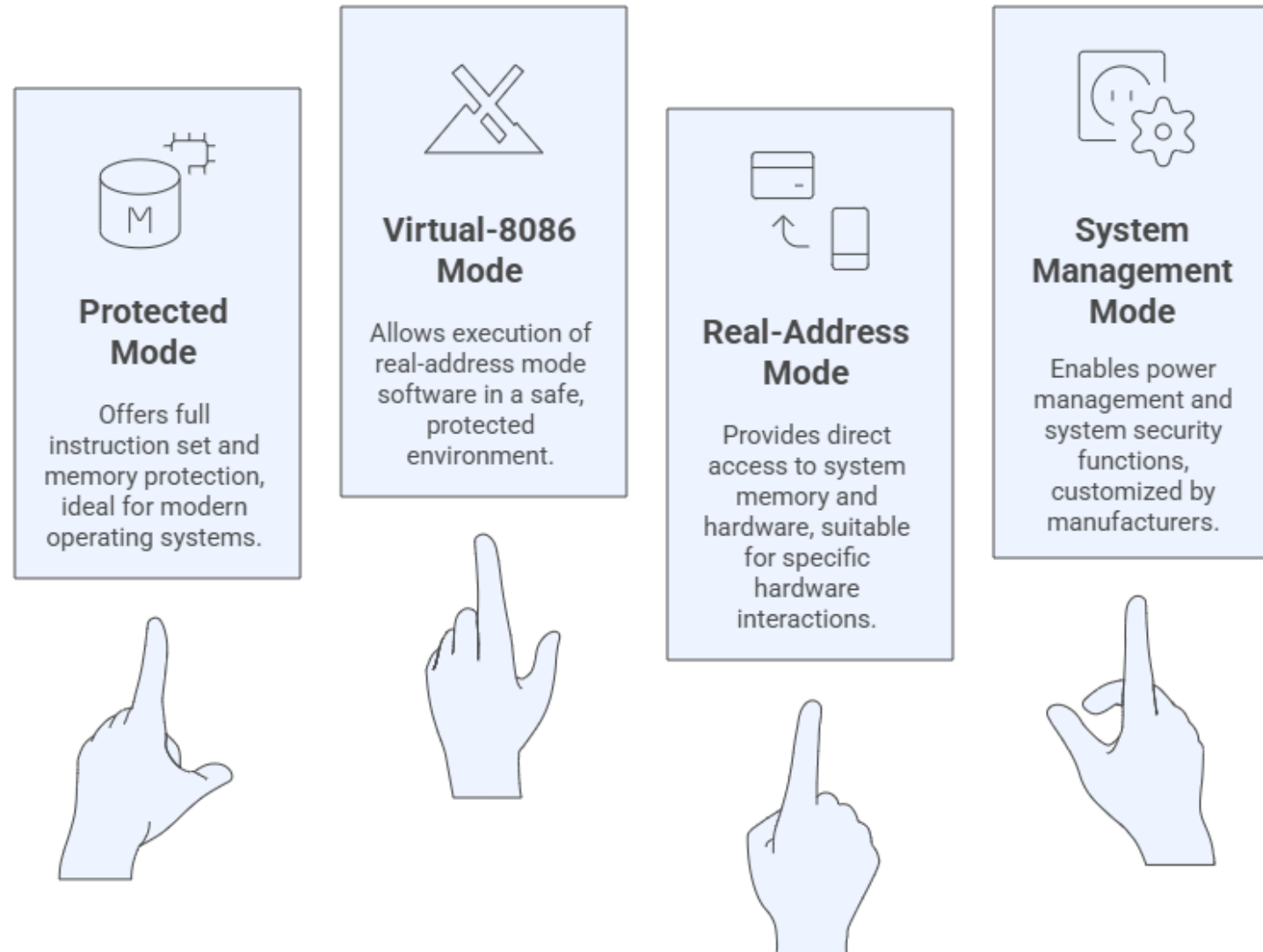


Memory Read Sequence

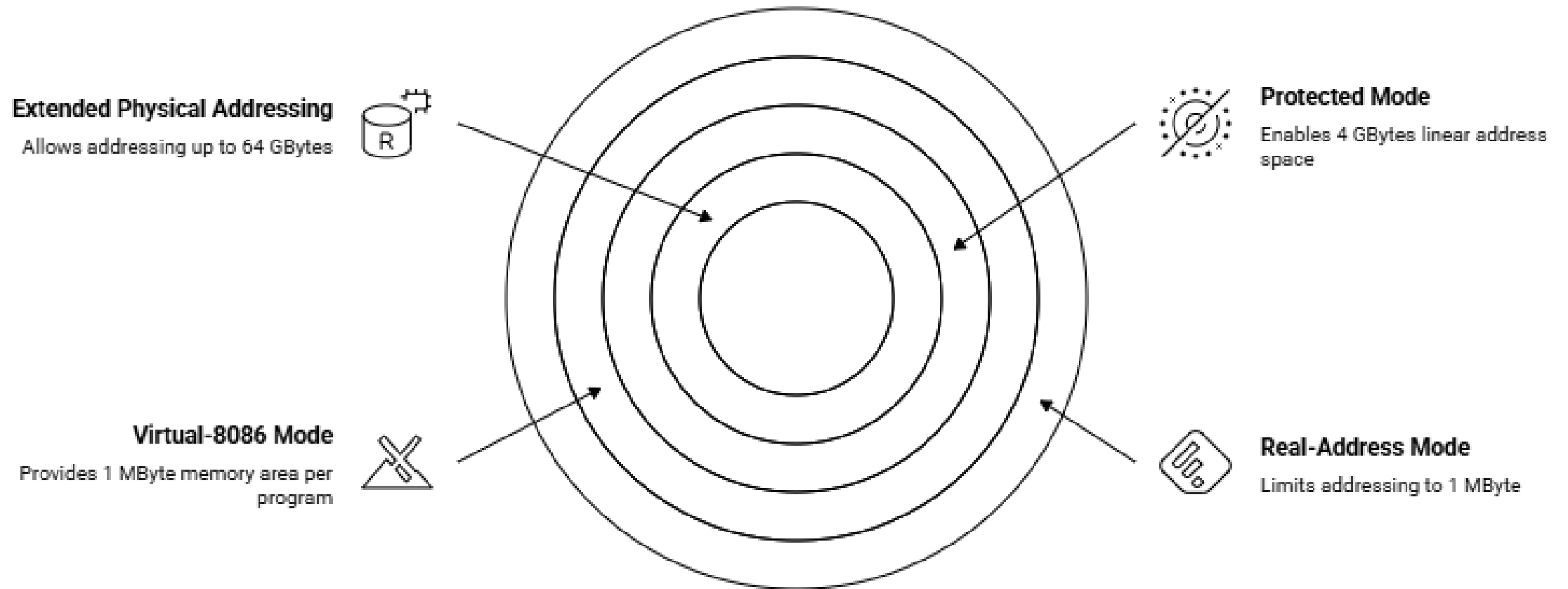




Which x86 processor mode should be used?



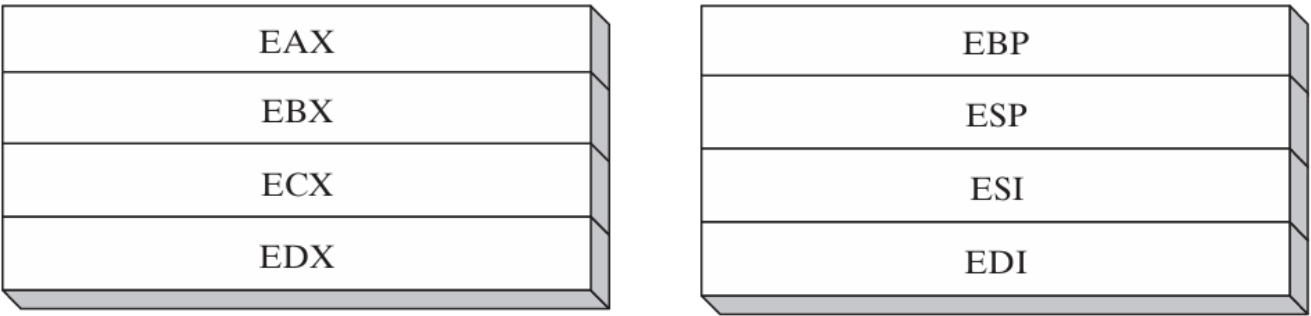
Memory Addressing Capabilities



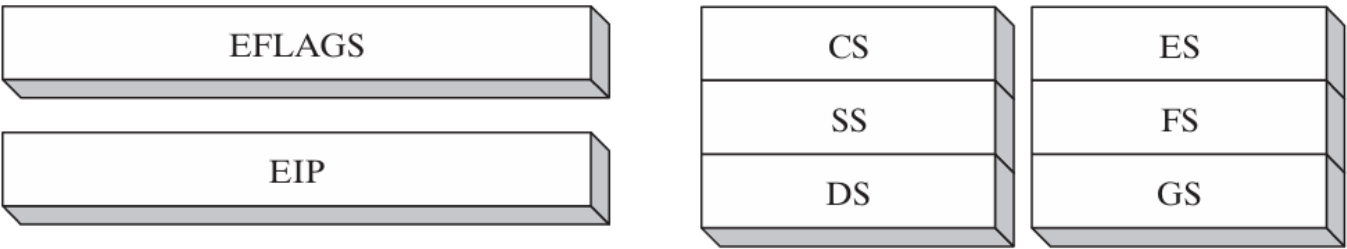
32-Bit x86 Processors

FIGURE 2-3 Basic program execution registers.

32-Bit General-Purpose Registers



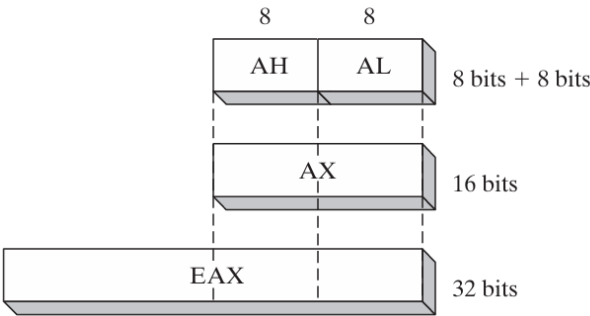
16-Bit Segment Registers



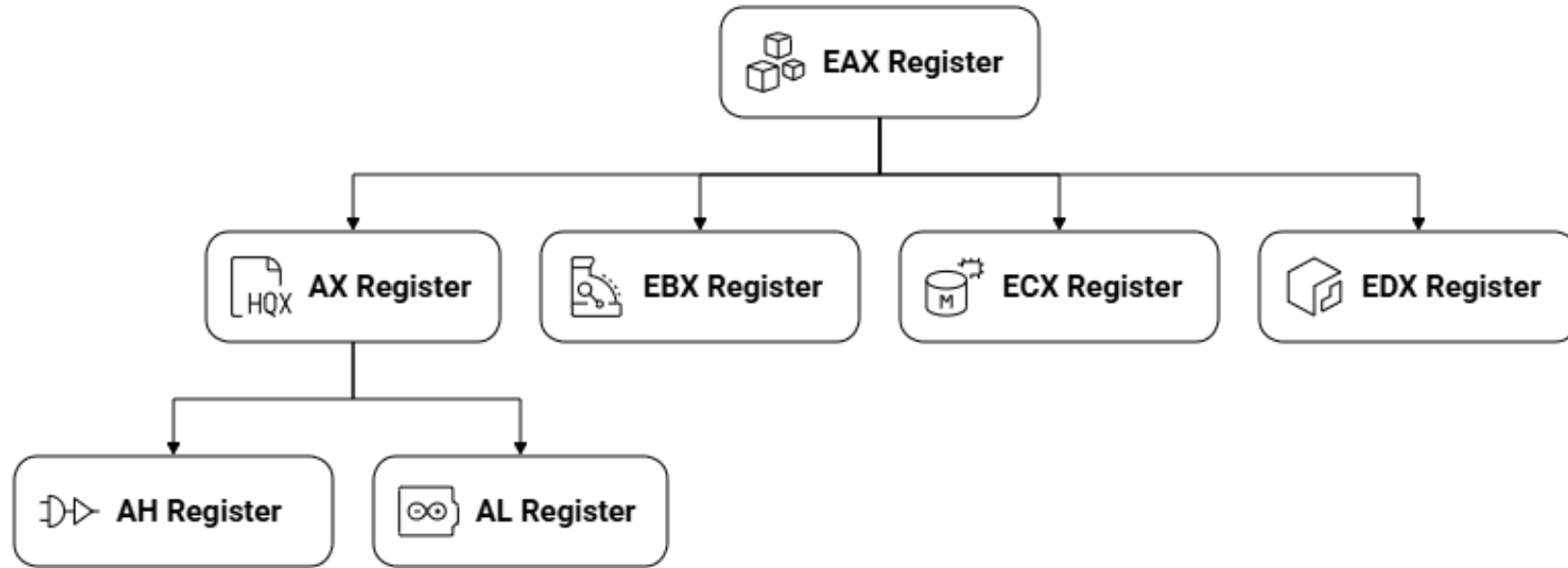
32-Bit	16-Bit	8-Bit (High)	8-Bit (Low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL

32-Bit	16-Bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP

FIGURE 2-4 General-purpose registers.

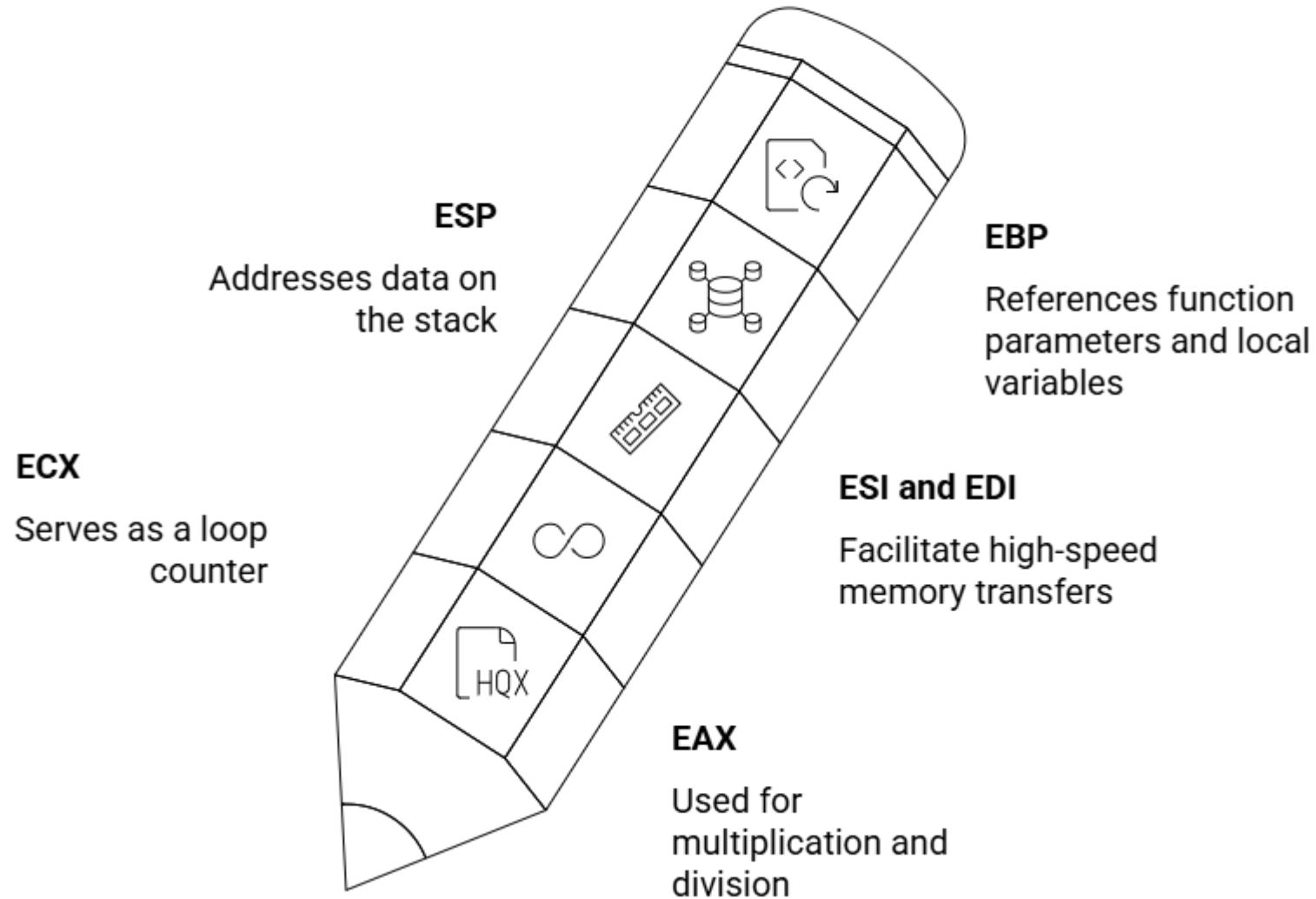


General-Purpose Registers and Their Subdivisions

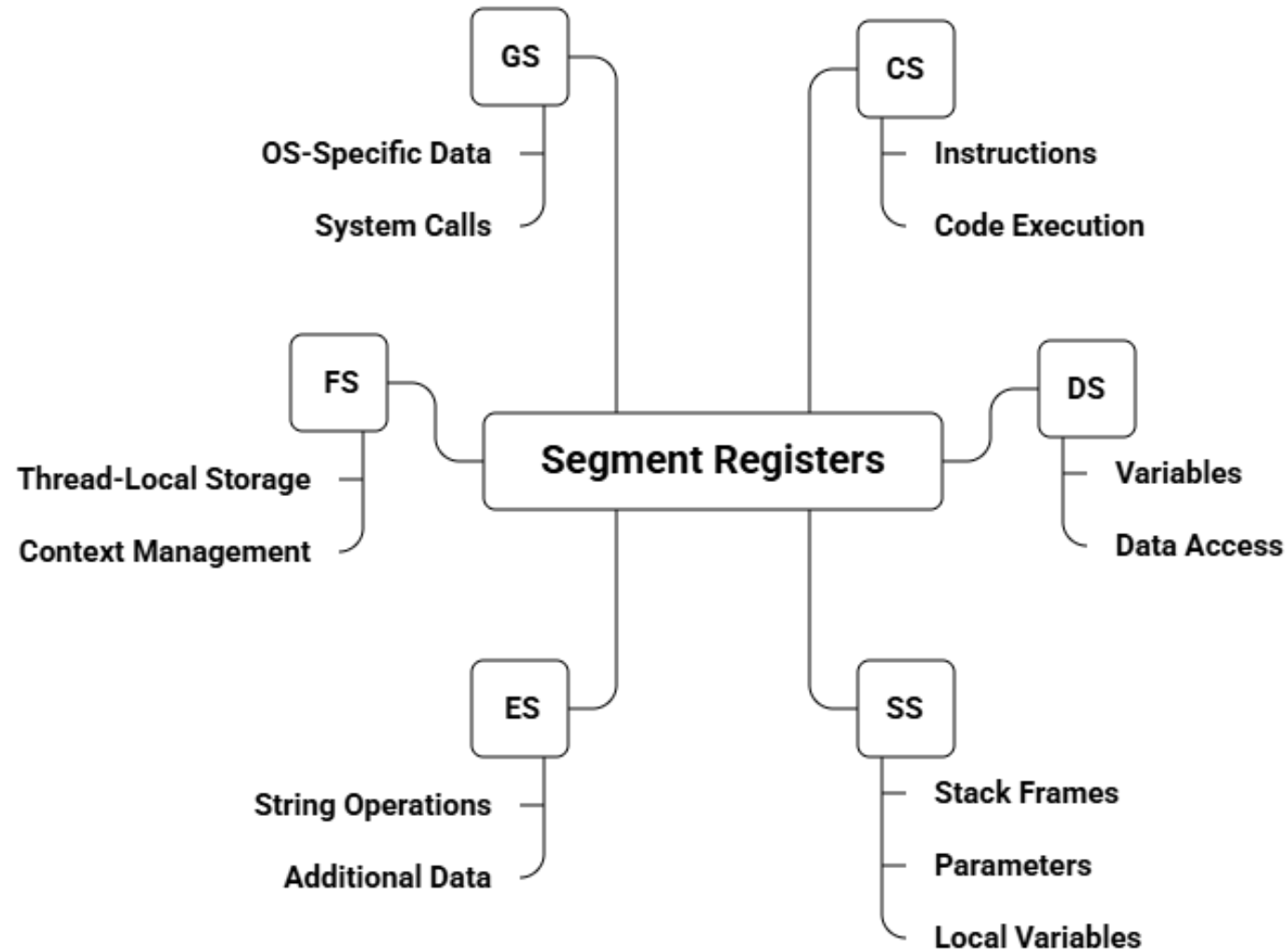


Made with  Napkin

Specialized Register Functions



x86 Segment Registers and Their Functions



EIP Register Explained

What is the EIP register?

It holds the address of the next instruction for the CPU to execute.

How does it work?

It updates automatically as instructions are executed, moving sequentially.

Can it be modified?

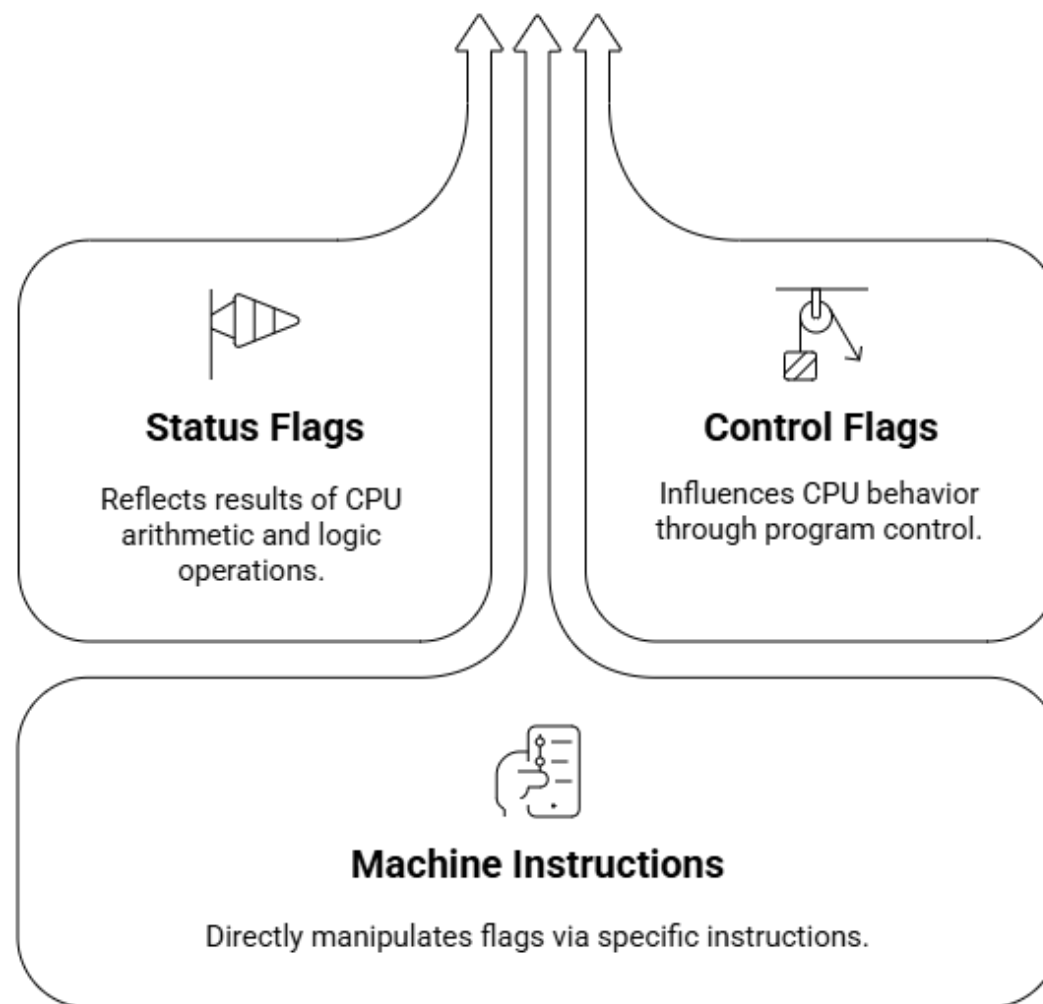
Yes, instructions like JMP, CALL, RET, or conditional jumps can change EIP, causing the program to branch.



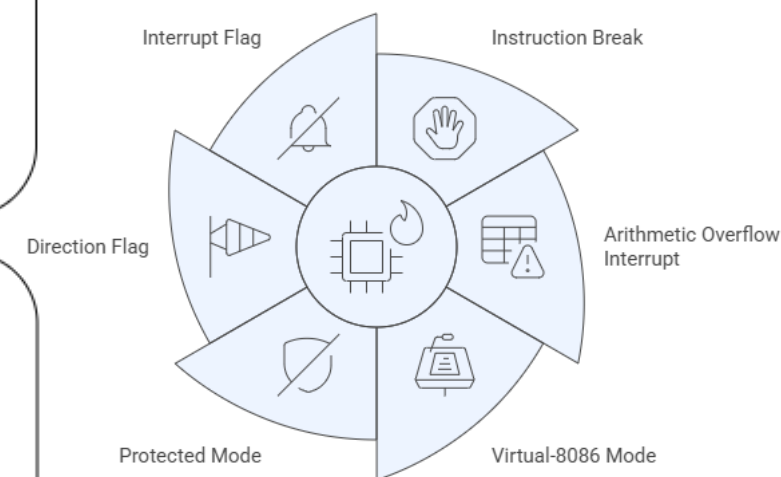
EFLAGS Register Flags and CPU Control

Comparison of CPU Status Flags

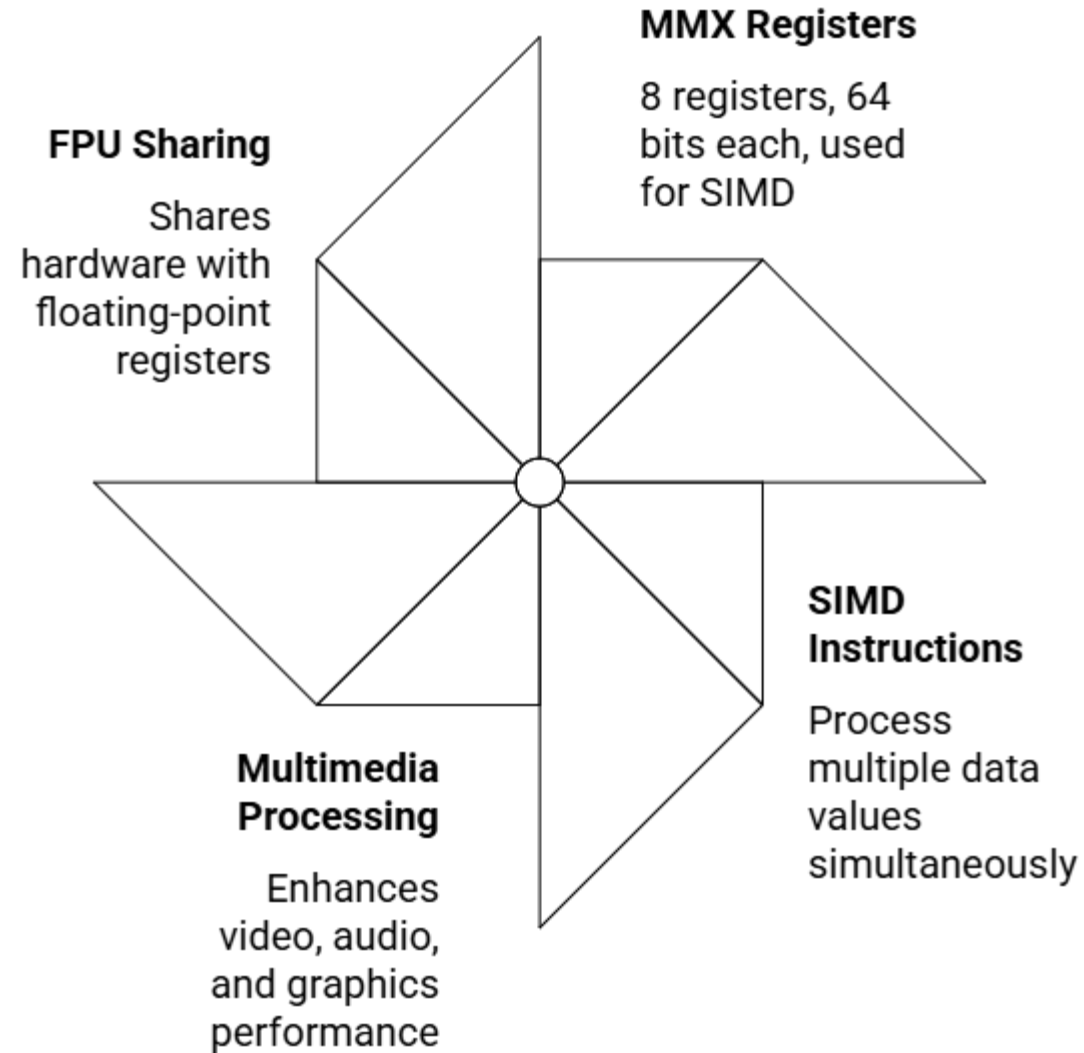
Status Flag	Description
Carry Flag (CF)	Unsigned result too large for destination
Overflow Flag (OF)	Signed result too large/small for destination
Sign Flag (SF)	Result is negative
Zero Flag (ZF)	Result is zero
Auxiliary Carry Flag (AC)	Carry from bit 3 to bit 4 (8-bit)
Parity Flag (PF)	Even number of 1 bits in result

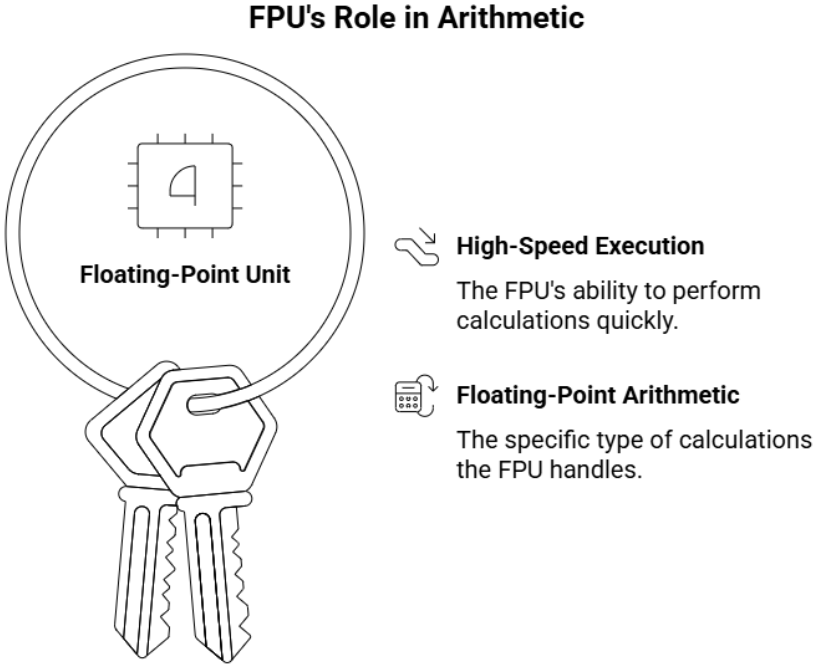


CPU Control Flags Overview



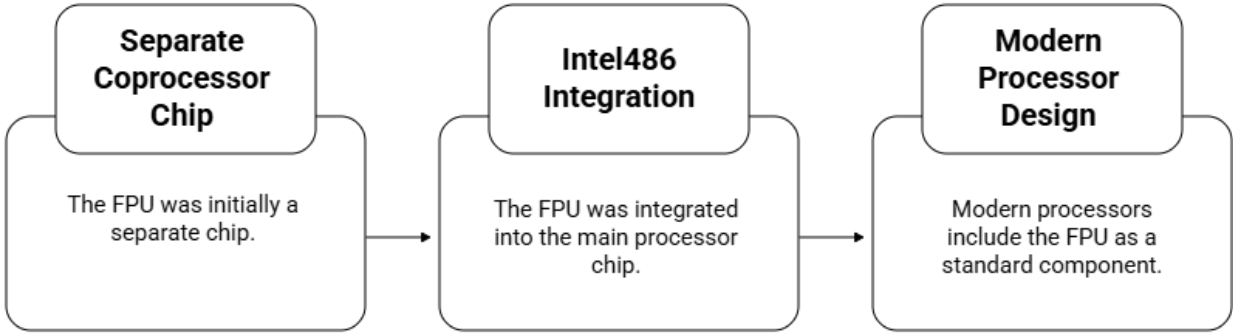
MMX Technology Overview





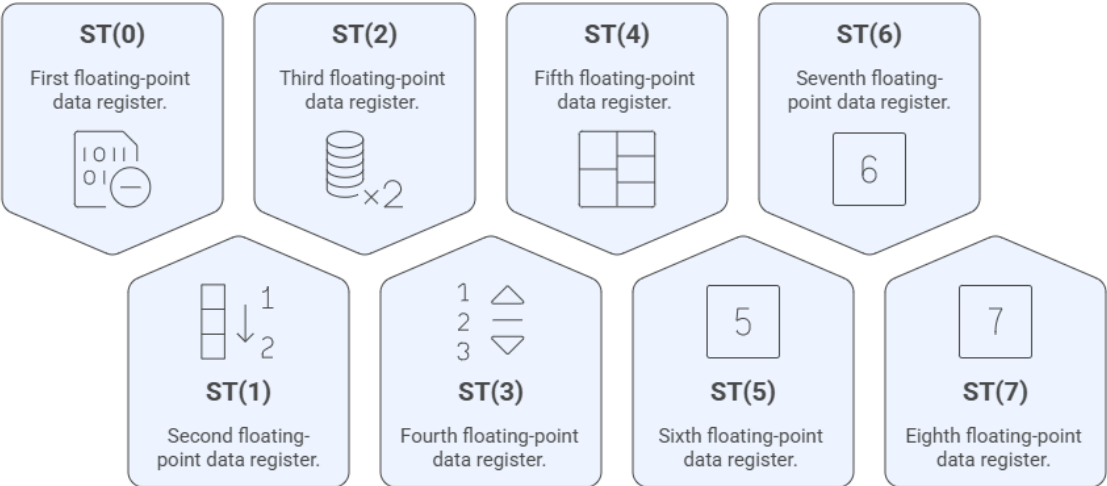
Made with Napkin

Evolution of Floating-Point Unit Integration



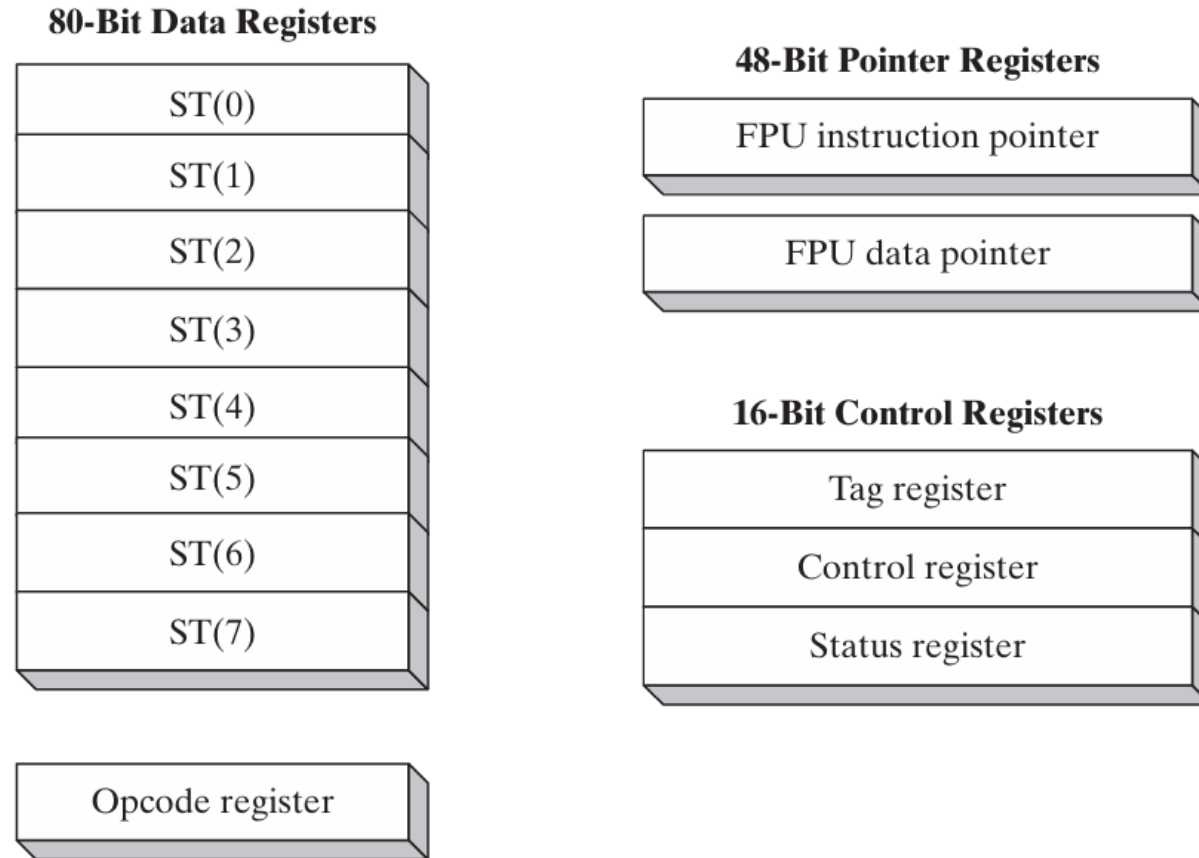
Made with Napkin

FPU Data Registers



Made with Napkin

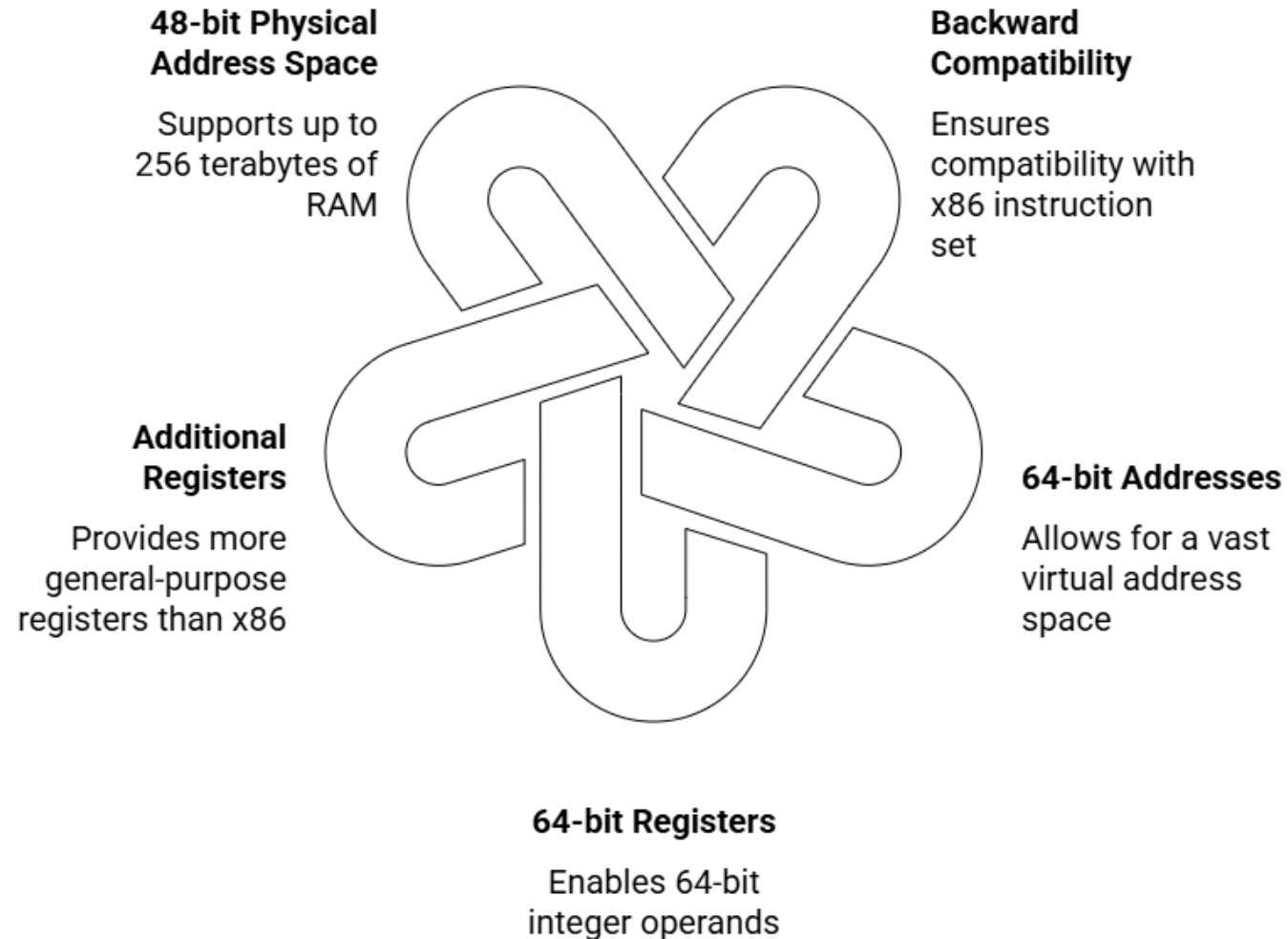
FIGURE 2-5 Floating-point unit registers.



x86 Processor Modes Comparison

Characteristic	Real-Address Mode	Protected Mode	Virtual-8086 Mode
 Memory Access	Direct access to any location	Restricted access, memory protection	Simulates real mode within protected
 Memory Limit	1 MB	4 GB per process	1 MB virtual space
 Program Execution	Single program at a time	Multiple programs simultaneously	Multiple virtual machines
 OS Support	MS-DOS, Windows 95/98	MS-Windows, Linux	Windows NT/2000/XP (for DOS)
 Hardware Access	Direct access	Restricted access	Limited, some programs may not run

Key Features of x86-64 Processors



64-Bit Operation Modes - IA-32e

Compatibility Mode

Allows running existing 16-bit and 32-bit applications without recompilation, but does not support 16-bit Windows and DOS applications.

64-Bit Mode

Enables 64-bit instruction operands and uses the 64-bit linear address space, suitable for native 64-bit applications.



32-bit vs 64-bit Processor Differences

Feature	32-bit	64-bit
Address Size	32 bits	48 bits (actual) / 64 bits (theoretical)
General Purpose Registers	Eight	Sixteen
Floating-Point Registers	Eight 80-bit	Eight 80-bit
Status Flags Register	32-bit (EFLAGS)	64-bit (RFLAGS)
Instruction Pointer	32-bit (EIP)	64-bit (RIP)
MMX Registers	Eight 64-bit	Eight 64-bit
XMM Registers	Eight 128-bit	Sixteen 128-bit

General Purpose Registers

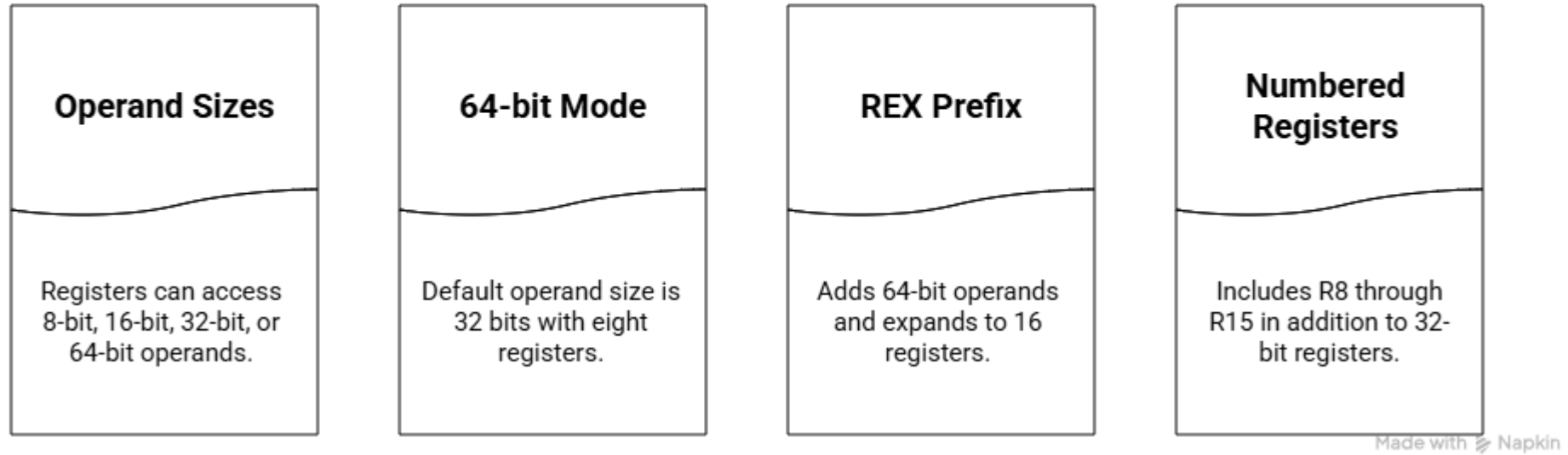


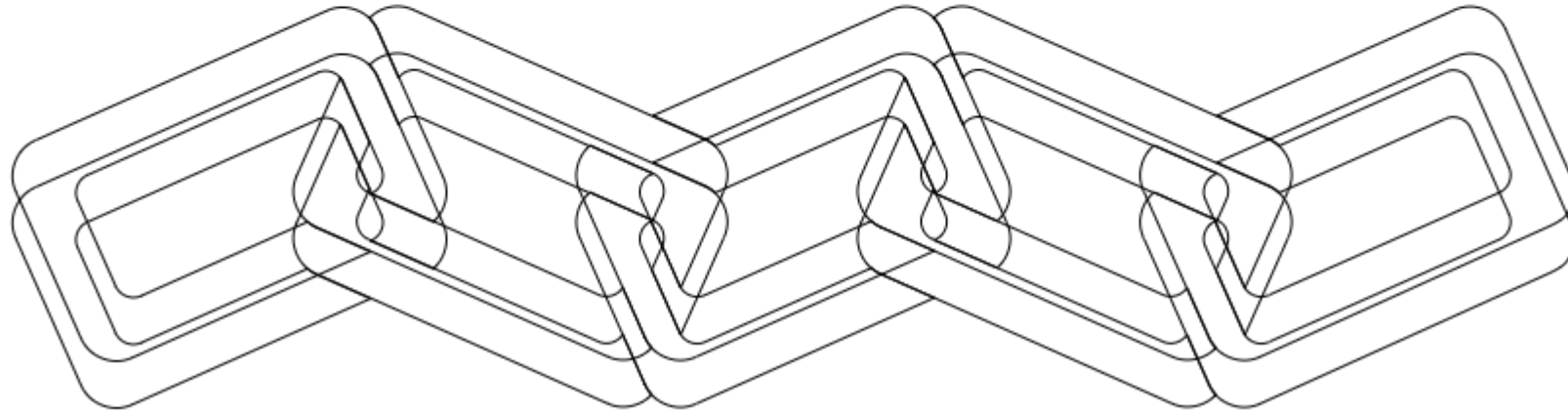
Table 2-1 Operand Sizes in 64-Bit Mode When REX Is Enabled.

Operand Size	Available Registers
8 bits	AL, BL, CL, DL, SIL, BPL, SPL, R8L, R9L, R10L, R11L, R12L, R13L, R14L, R15L
16 bits	AX, BX, CX, DX, DI, SI, BP, SP, R8W, R9W, R10W, R11W, R12W, R13W, R14W, R15W
32 bits	EAX, EBX, ECX, EDX, EDI, ESI, EBP, ESP, R8D, R9D, R10D, R11D, R12D, R13D, R14D, R15D
64 bits	RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP, R8, R9, R10, R11, R12, R13, R14, R15

Components of a typical x86 computer

Components of a Typical x86 Computer

Motherboard Configuration	Memory	I/O Ports	Device Interfaces	Assembly Language I/O
The physical layout and components of the motherboard	The system's storage and retrieval mechanisms	Interfaces for connecting external devices	Standards for communication between devices	Programming techniques for interacting with hardware

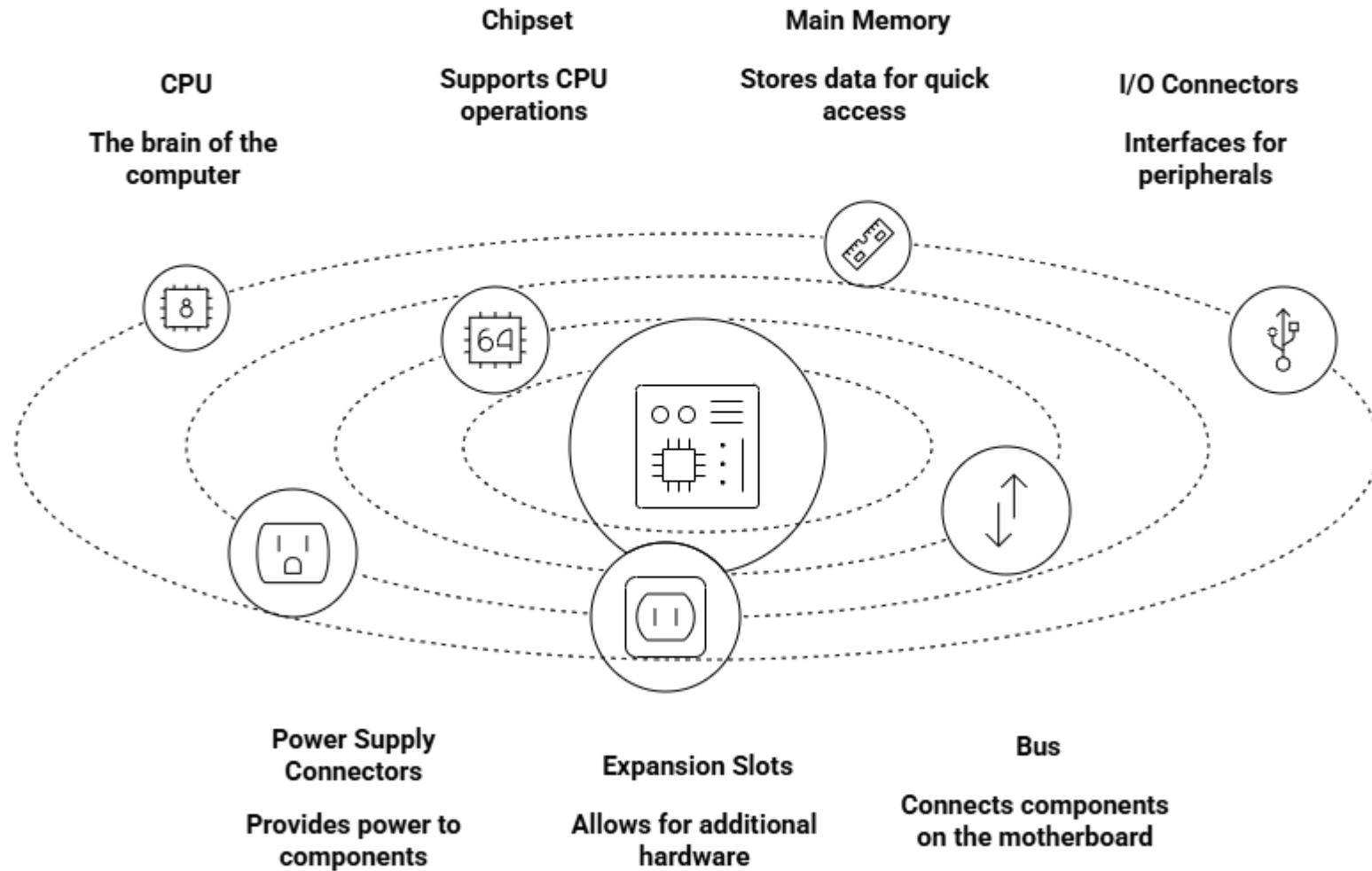


Made with  Napkin

Assembly programming

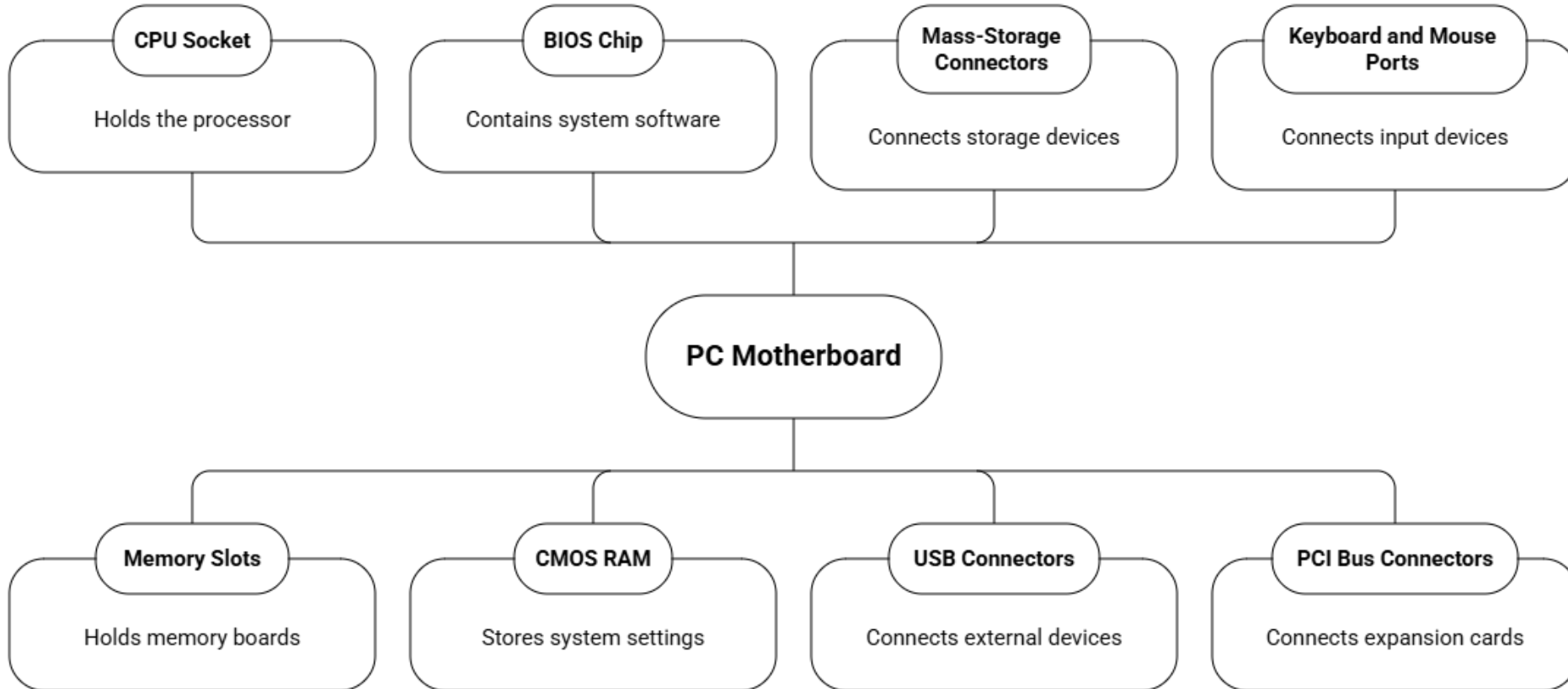
Components of a typical x86 computer

Anatomy of a Motherboard



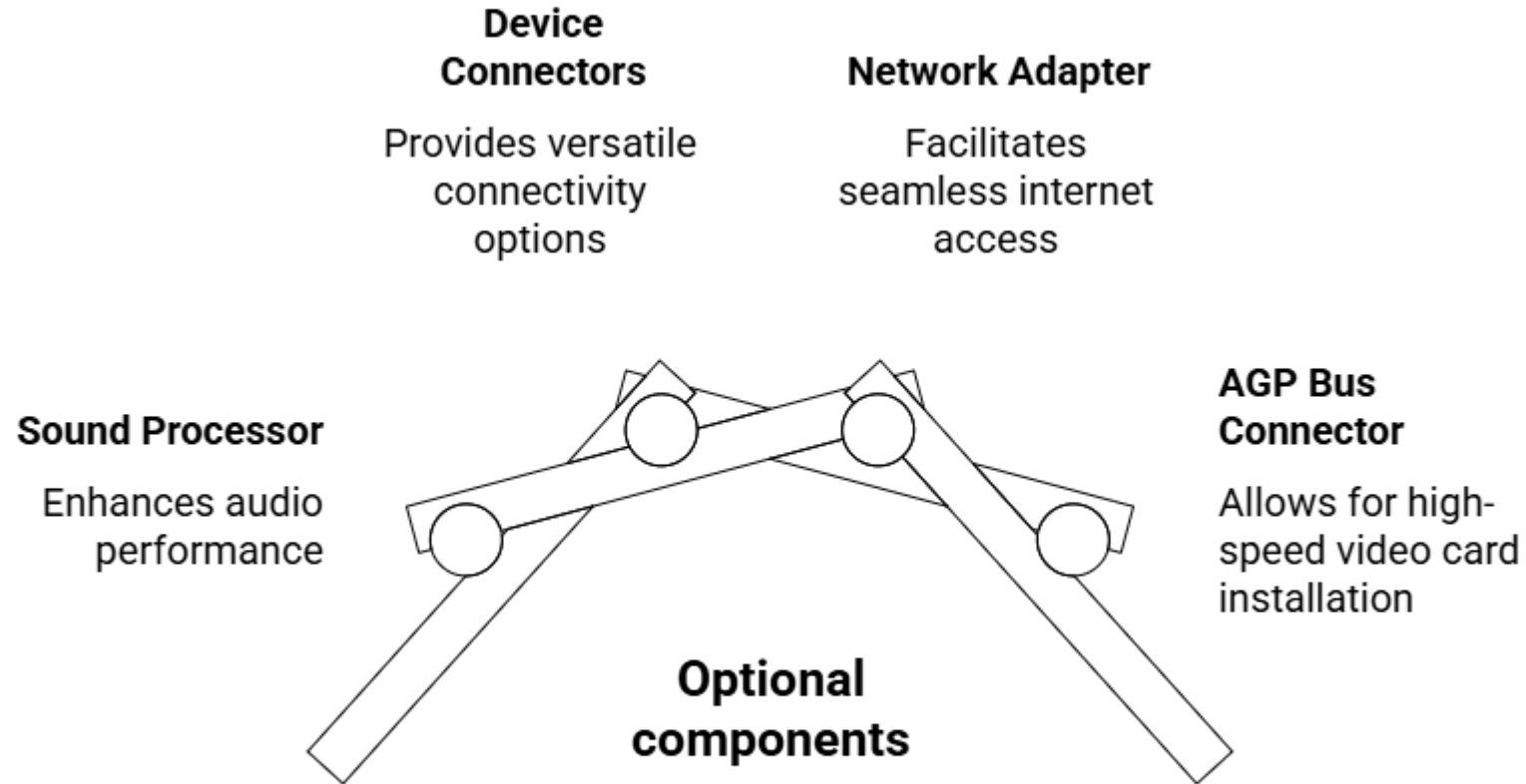
Components of a typical x86 computer

Anatomy of a PC Motherboard



Made with  Napkin

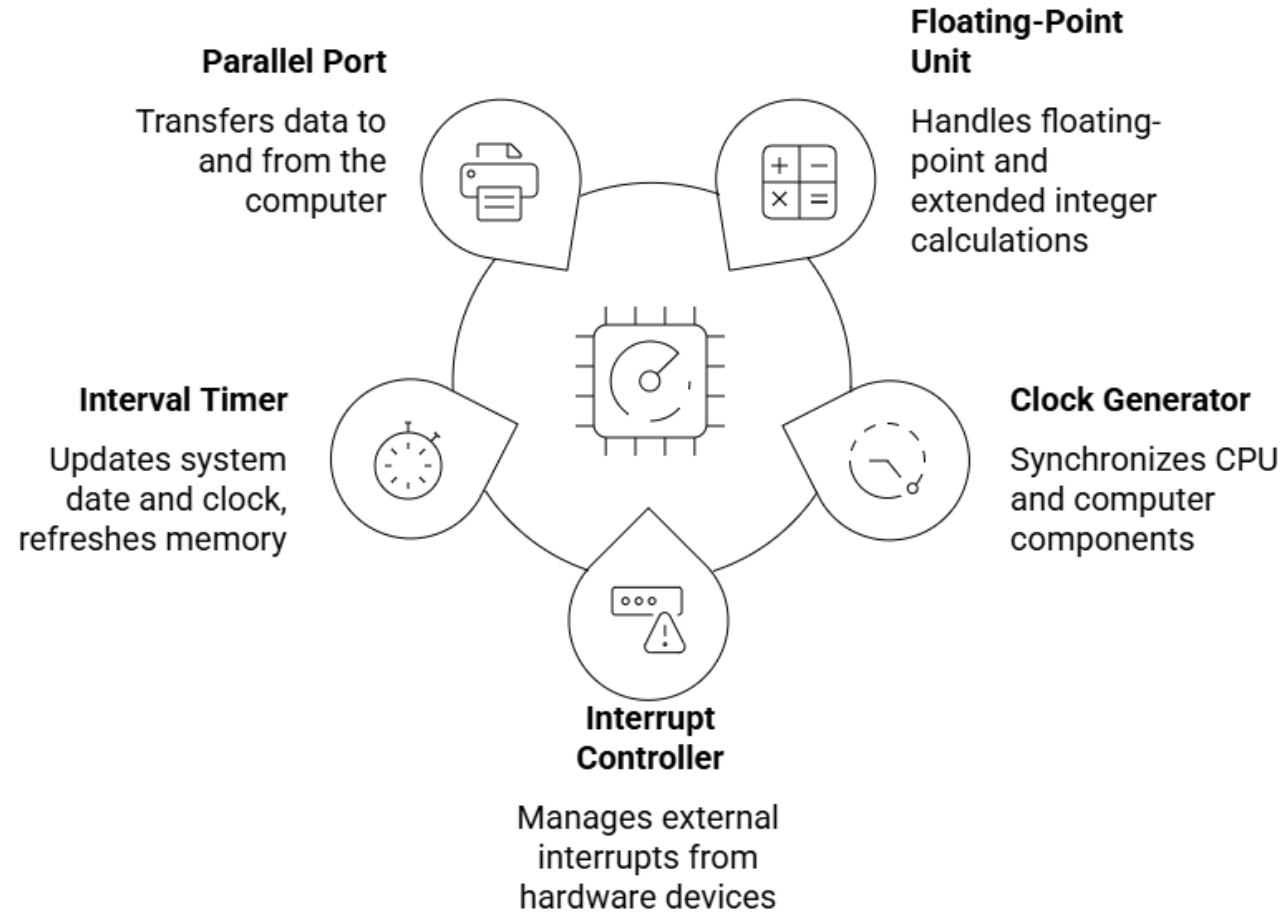
Components of a typical x86 computer



Made with  Napkin

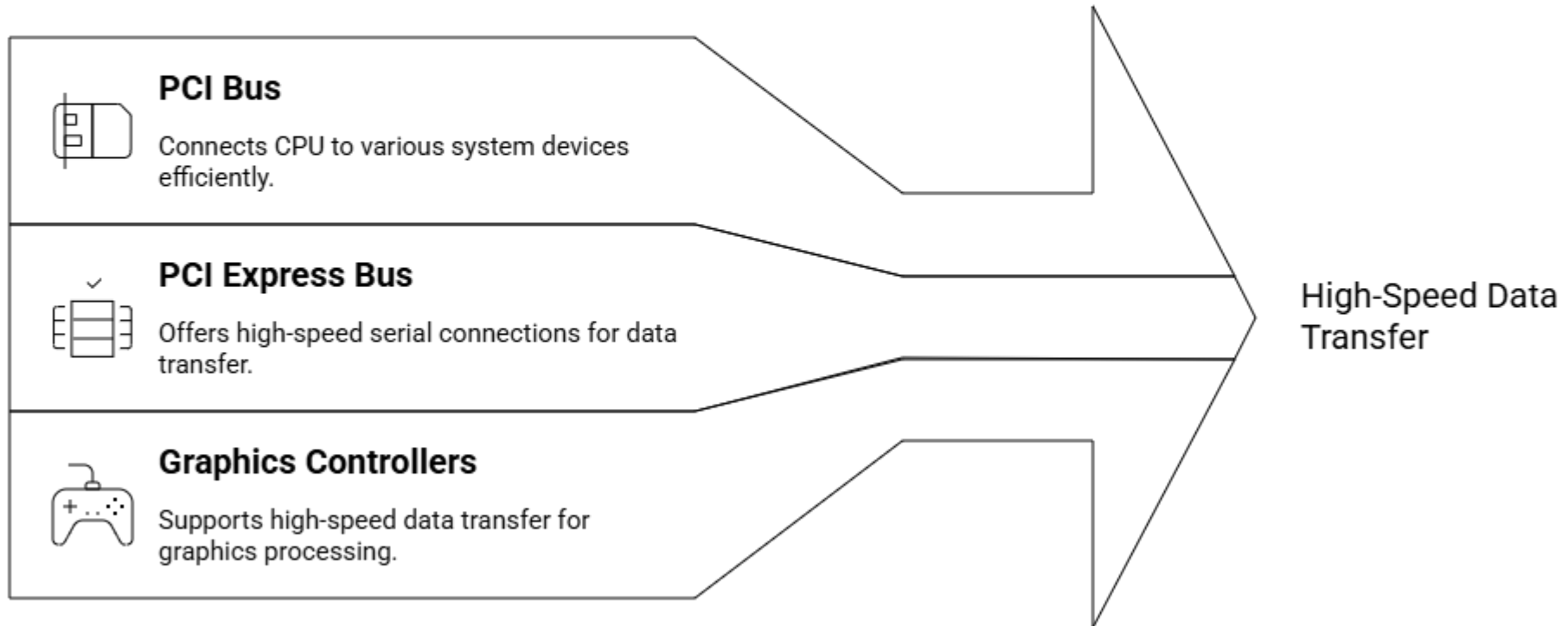
Components of a typical x86 computer

Components Supporting Processor Functionality



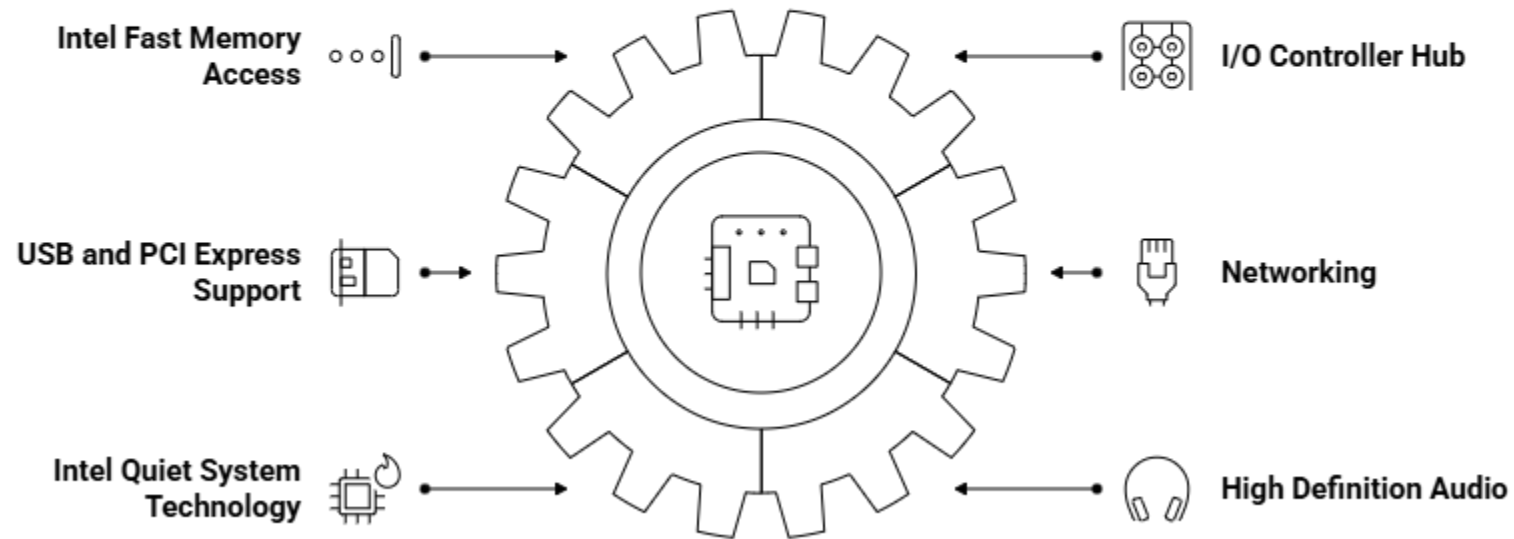
Made with  Napkin

PCI and PCI Express Bus Architectures



Components of a typical x86 computer

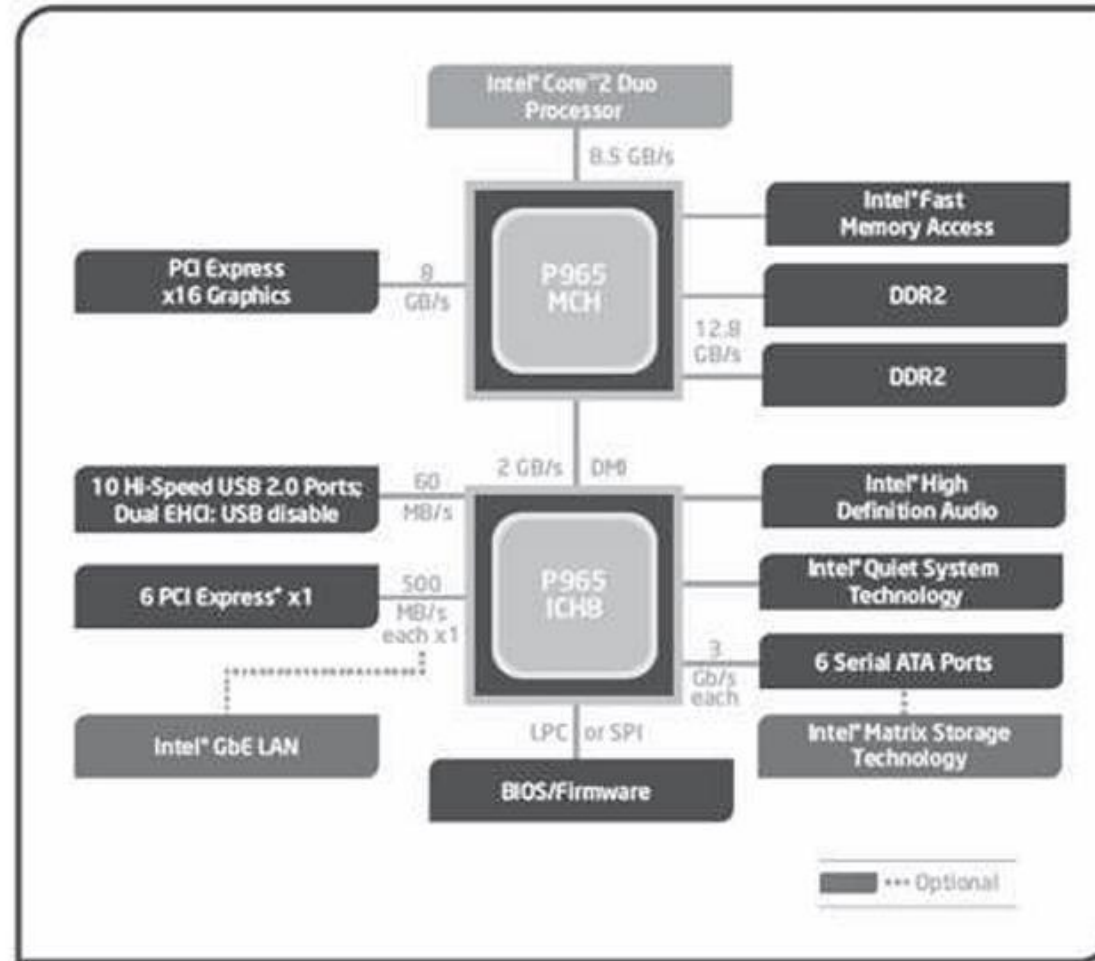
Intel P965 Express Chipset Overview



Made with  Napkin

Components of a typical x86 computer

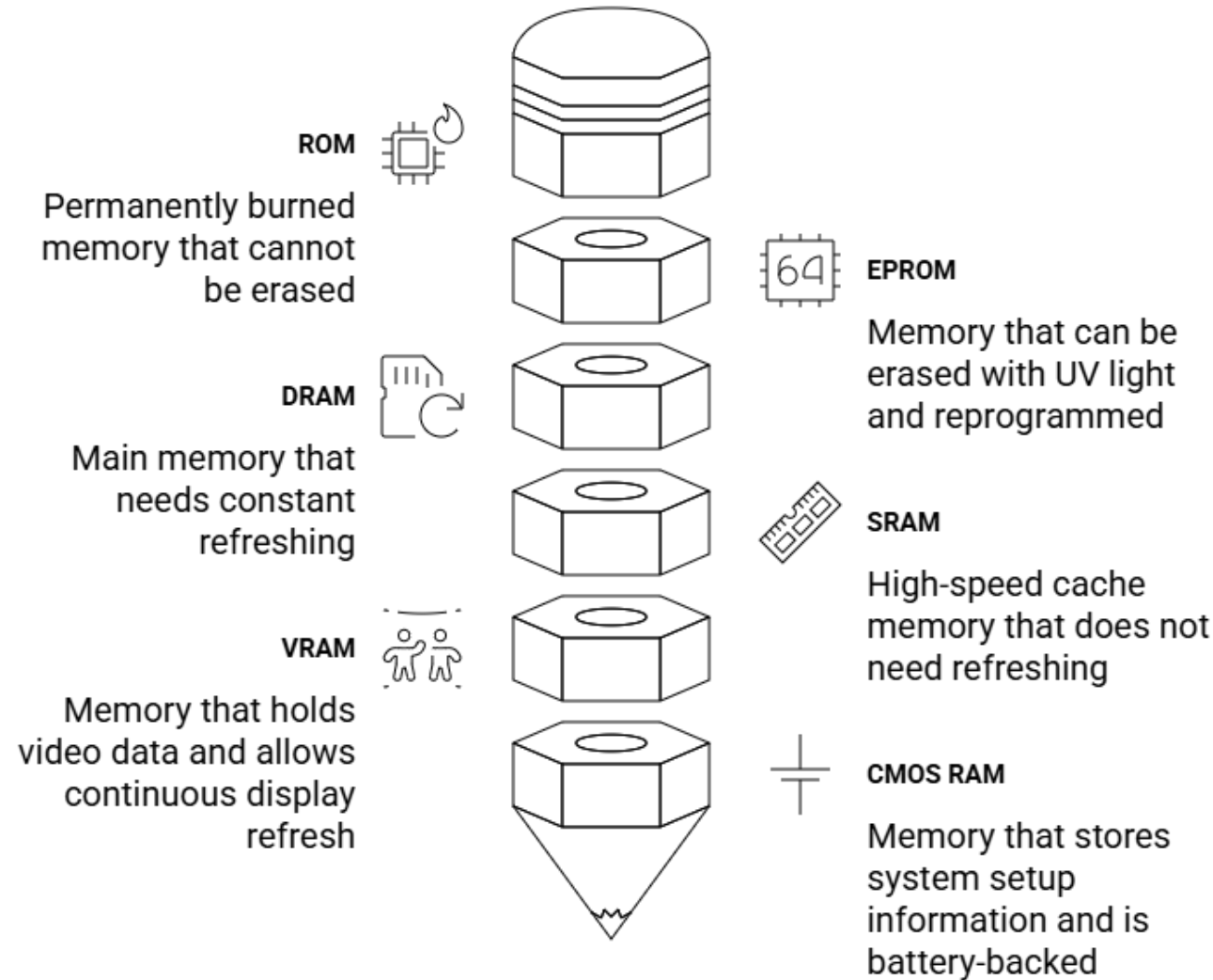
FIGURE 2–6 Intel 965 express chipset block diagram.



Source: The Intel P965 Express Chipset (product brief),
© 2006 by Intel Corporation, used by permission.

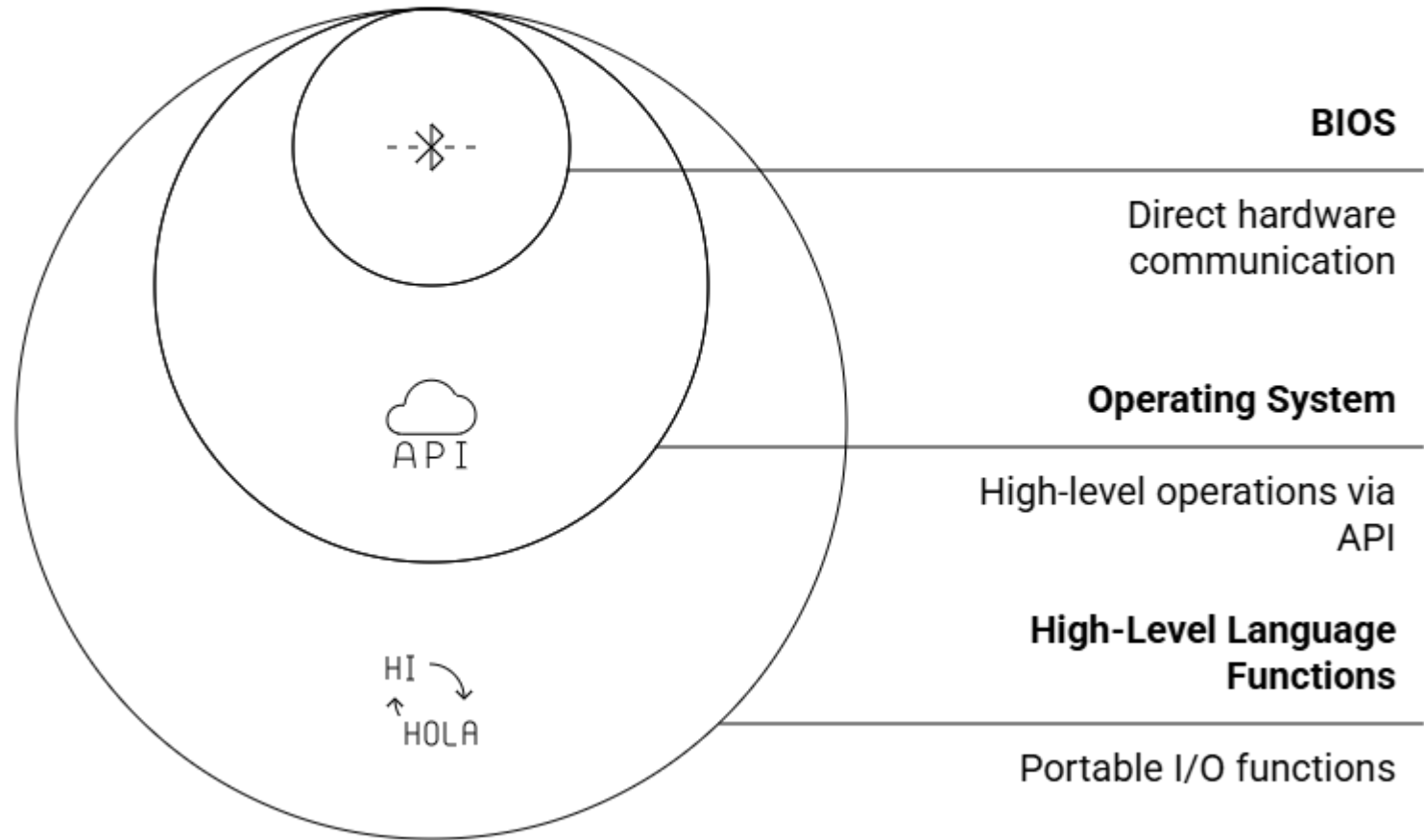
Components of a typical x86 computer

Memory Types in Intel Systems

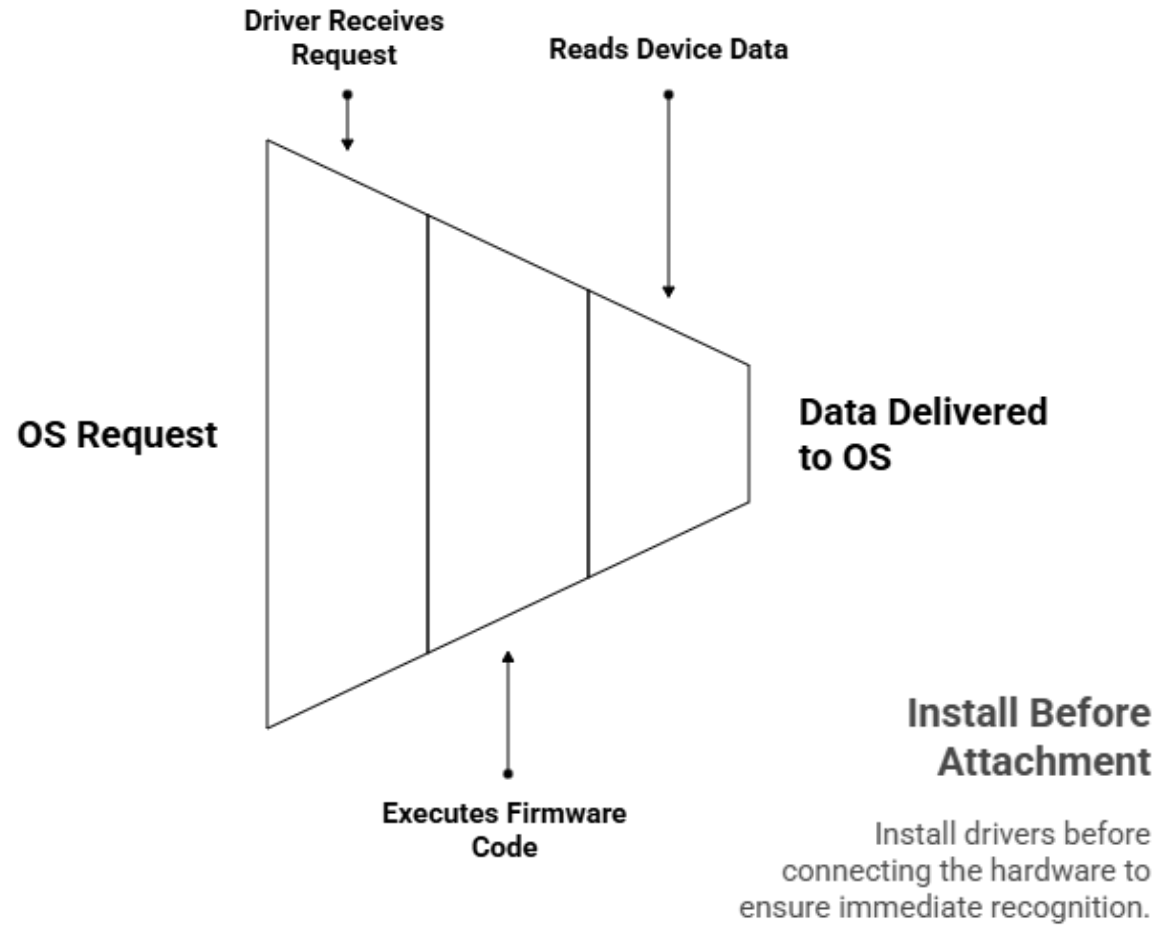


Components of a typical x86 computer

Levels of I/O Access



Device Driver Communication Process



How to install device drivers?



Install After Attachment

Install drivers after the device is connected, allowing the OS to identify and install automatically.

Displaying a String on Screen

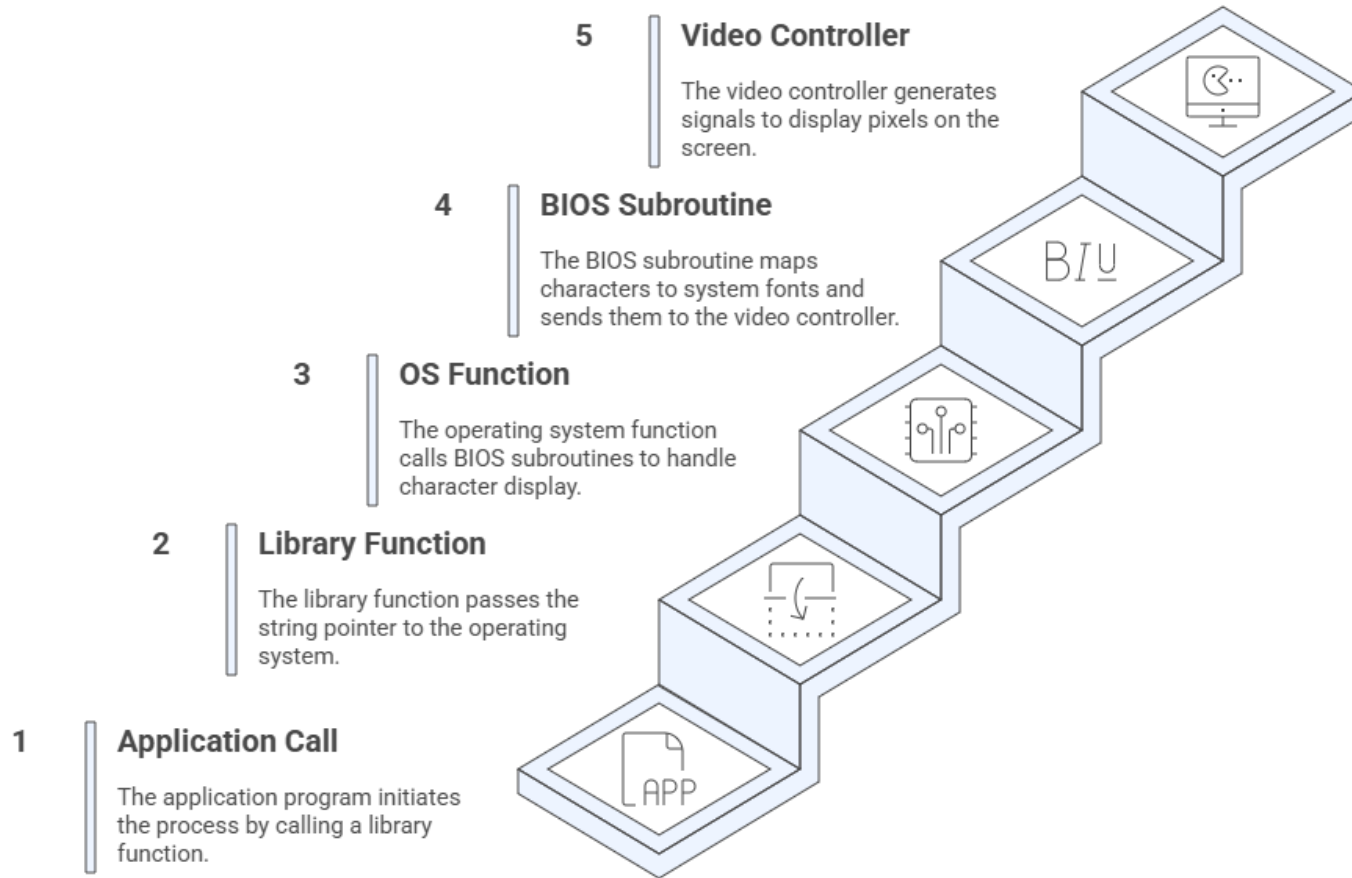
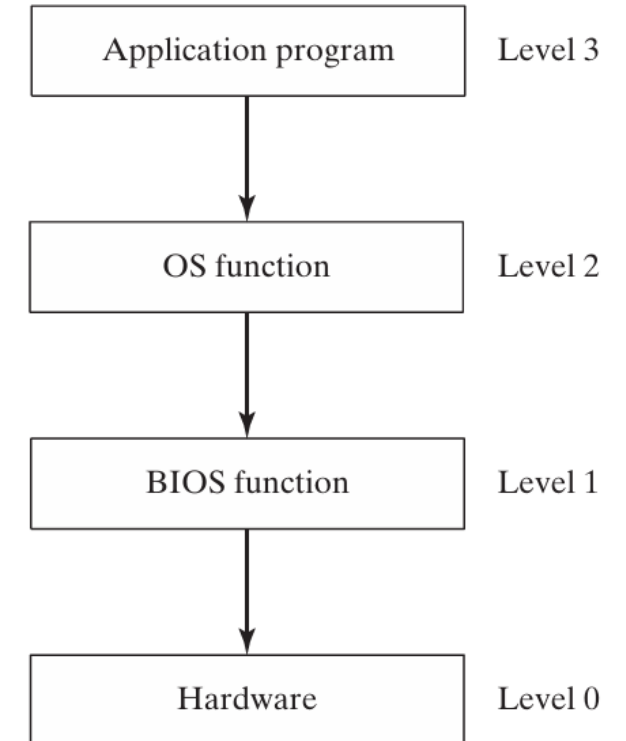
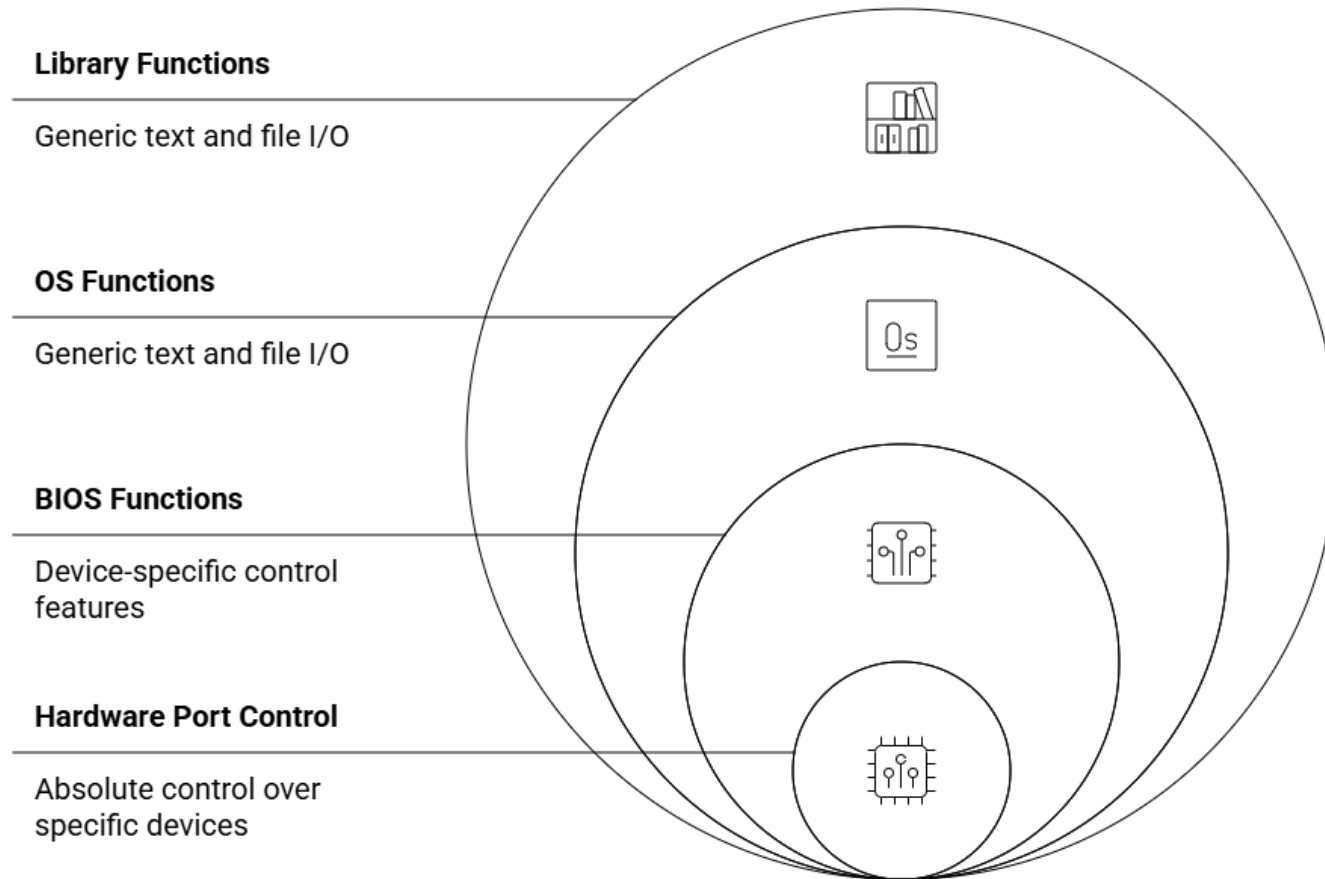


FIGURE 2-7 Access levels for input–output operations.

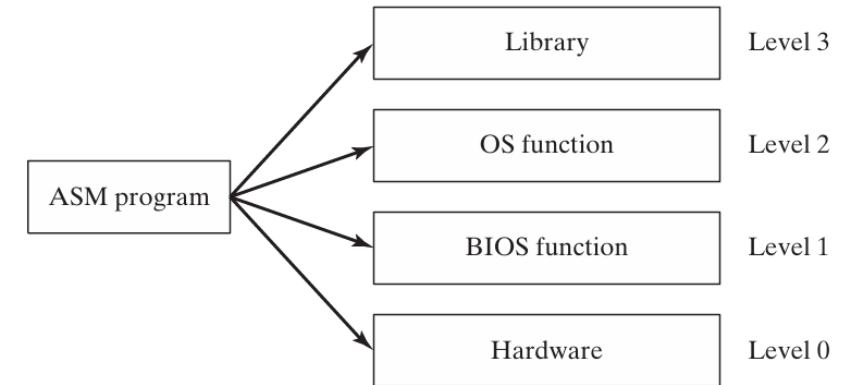


Assembly Language I/O Levels



Made with  Napkin

FIGURE 2–8 Assembly language access levels.



Input-output system

2 x86 Processor Architecture 32

2.1 General Concepts 33

- 2.1.1 Basic Microcomputer Design 33
- 2.1.2 Instruction Execution Cycle 34
- 2.1.3 Reading from Memory 36
- 2.1.4 Loading and Executing a Program 36
- 2.1.5 Section Review 37

2.2 32-Bit x86 Processors 37

- 2.2.1 Modes of Operation 37
- 2.2.2 Basic Execution Environment 38
- 2.2.3 x86 Memory Management 41
- 2.2.4 Section Review 42

2.3 64-Bit x86-64 Processors 42

- 2.3.1 64-Bit Operation Modes 43
- 2.3.2 Basic 64-Bit Execution Environment 43

2.4 Components of a Typical x86 Computer 44

- 2.4.1 Motherboard 44
- 2.4.2 Memory 46
- 2.4.3 Section Review 46

2.5 Input–Output System 47

- 2.5.1 Levels of I/O Access 47
- 2.5.2 Section Review 49

Review questions

Github upload

Presentation subject

- Motherboard
- CPU & GPU
- Main memory & SSD
- Keyboard, Mouse, Monitor
- PC building